CSC 4320/6320 – Chapter 2: Processes, Threads, & Scheduling (Slides + Speaker Notes)

Slide 1

Slide Content:
Operating Systems
CSC4320, CSC6320
FALL 2026
Dr. Roya Hosseini
Chapter 2: Processes, Threads, Resources, and CPU Scheduling
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 2

Slide Content:
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Copyright and Use Notice
These slides were prepared by Dr. Roya Hosseini for
CSC4320, CSC6320: Operating Systems at Georgia State University.
These materials are provided for instructional use only.
Unauthorized copying, posting, sharing, selling, or redistribution of these materials is not permitted without written permission from the instructor.
(c) Dr. Roya Hosseini, Georgia State University, Fall 2026
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
(No additional speaker notes)

Slide 3

Slide Content:
Chapter 2: Processes, Threads, Resources, and CPU Scheduling
- 2.1 Process Concept
- 2.2 Process States and the PCB
- 2.3 Process Operations
- 2.4 Interprocess Communication
- 2.5 Thread Fundamentals
- 2.6 Multithreading Models and APIs
- 2.7 Context Switching
- 2.8 CPU-Scheduling Foundations
- 2.9 Scheduling Criteria
- 2.10 Scheduling Algorithms
- 2.11 Advanced Scheduling

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
(No additional speaker notes)

Slide 4

Slide Content:
CHAPTER MAP

By the end of this chapter, you can trace execution from process creation to CPU selection
04
01
Execution units
Processes, PCBs, resources, and threads
02
Coordination
Creation, IPC, lifecycle, and context switching
03
Scheduling
Bursts, criteria, dispatcher, and core algorithms
04
Scale + deadlines
Multicore placement and real-time policies
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 5

Slide Content:
SECTION 2.1
Process Concept
A program becomes a process when the operating system gives it an execution context
2.1 / 11
05
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 6

Slide Content:
2.1 PROCESS CONCEPT
A program is passive; a process is executing
06
PROGRAM
Executable instructions stored on disk
No current CPU state
Can be launched many times
PROCESS
A running instance of a program
Has a PID, address space, and CPU state
Owns or references resources
One program can produce many independent processes.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

PID stands for Process Identifier.
It is a unique number assigned by the operating system to each running process. The OS uses PIDs to identify, manage, and control processes.
PID -¿ Process
1 -¿ systemd or another init process
850 -¿ Web browser
1024 -¿ Text editor =¿ kill 1024
1260 -¿ Terminal
A daemon is a program that runs in the background and provides a system or network service, usually without direct user interaction.

Slide 7

Slide Content:
2.1 PROCESS CONCEPT
A process address space has regions with different roles
07
STACK ↓
function calls, parameters, local variables
HEAP ↑
dynamic allocation; grows toward free space
DATA
global and static variables
TEXT
executable machine instructions
Higher virtual addresses near the stack
Lower virtual addresses near the text segment
• Stack grows downward
• Heap grows upward
• Free space separates them
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Free virtual-address space **Speaker Notes (Under Slide):**

Which regions are shared by threads in one process? Expected: text, data, and heap are shared; each thread has its own stack
Text segment: Contains the program's executable machine instructions. It is normally read-only and may be shared by multiple processes running the same program.
Data segment: Contains global and static variables.
- Initialized variables are stored in the data section.
- Uninitialized or zero-initialized variables are generally stored in the BSS section.
Heap: Stores dynamically allocated memory requested through functions such as malloc(), calloc(), or new. It traditionally grows toward higher addresses.
Stack: Stores function-call frames, parameters, return addresses, and local variables. It traditionally grows toward lower addresses.
Free space: The unused virtual-address range between the heap and stack allows both regions to expand.

Slide 8

Slide Content:
(Visual Slide)

```c
int global_value = 10;        // Data segment

int main(void) {
    int local_value = 5;      // Stack
    int *number = malloc(sizeof(int));  // Allocated object is on heap
}
```

Speaker Notes (Under Slide):
malloc(sizeof(int)) =¿ requests that amount of memory from the heap.
malloc() =¿ returns the address of the allocated memory.

Slide 9

Slide Content:
2.1 PROCESS CONCEPT

A process is more than the instructions it executes
08
CPU STATE
PC, registers, flags
MEMORY
code, data, heap, stack
FILES
open file descriptors
IDENTITY
PID, user, group
COMMUNICATION + ACCOUNTING
pipes, signals, sockets; time, limits, usage
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

A process consists of executable code, its current execution state, its memory, its identity, and the resources assigned to it by the operating system.
PID: Unique process identifier
User ID: Identifies the user who owns the process
Group ID: Identifies the process owner's group
Code/text: Executable instructions
Data: Global and static variables
Heap: Dynamically allocated memory
Stack: Function calls, parameters, return addresses, and local variables
The OS maintains the process's open file descriptors. A file descriptor is a small integer representing an open file or another input/output resource.
Common examples:
0 – standard input
1 – standard output
2 – standard error

Slide 10

Slide Content:
2.1 PROCESS CONCEPT
PIDs and parent-child links organize running work
09
PROCESS IDENTIFIER
• Unique number within a running system
• Used by tools and system calls
• May be reused after termination
PARENT-CHILD RELATIONSHIP
• A creator process becomes the parent
• Children may inherit selected resources
• The OS records ancestry in a process tree
Identity tells the kernel which execution context an operation targets.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Process identifier
A PID is a number used to identify a process while it is running.
Every active process has a PID.
Tools and system calls use the PID to select a process.
After a process terminates and its information is removed, its PID may eventually be assigned to another process.
For example:
kill 2450
This sends a signal to the process currently identified by PID 2450.
A PID is therefore unique among currently active processes, but it is not permanently associated with one program.
Parent-child relationship
When one process creates another process:
The creator is the parent process.

The newly created process is the child process.
The child receives its own PID.
The OS stores the parent's PID as the child's PPID–Parent Process Identifier.
For example:
Terminal: PID 1200
Program: PID 1250, PPID 1200
The terminal process created the program, so the terminal is its parent.

Slide 11

Slide Content:
SECTION 2.2
Process States and the PCB
State describes what a process can do now; the PCB preserves everything needed to continue later
2.2 / 11
10
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

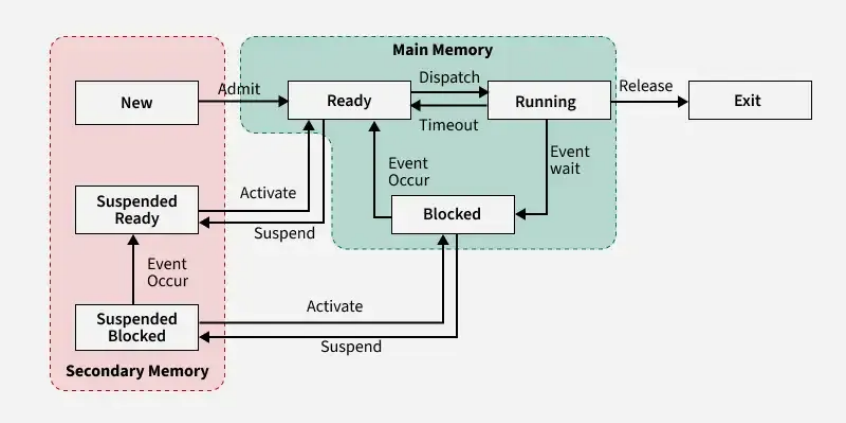
Slide 12

Slide Content:
2.2 STATES + PCB
One diagram connects process states to the events that move them
11
NEW
being created
READY
waiting for CPU
RUNNING
executing now
WAITING
blocked for event
TERMINATED
execution finished
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
admit
dispatch
exit
block
wake
preempt **Speaker Notes (Under Slide):**

Trace each arrow aloud. =¿waiting returns to ready–not directly to running–because the scheduler must select the process again.

Slide 13

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
https://www.geeksforgeeks.org/operating-systems/states-of-a-process-in-operating-systems/
The main process states are:
New: The process is in the creation phase. The program exists in secondary memory, and the OS prepares its PCB before loading it into main memory.
Ready: The process has been loaded into main memory and is prepared to run. It is waiting in the ready queue until the CPU becomes available.
Running: The CPU is currently executing the instructions of the process. Only one process can be in this state at a time on a single-processor system.
Blocked/Waiting: The process cannot continue because it is waiting for an event such as I/O completion, user input, or access to a locked resource. Once the event is complete, it moves back to the ready state.
Terminated: The process has completed its execution or has been stopped. The OS deletes its PCB and frees all resources allocated to it.
Suspend Ready: A ready process that has been swapped out of main memory and placed in secondary storage due to memory shortage. When brought back to main memory, it returns to the ready state.
Suspend Block: A blocked process that has been swapped out of main memory and placed in secondary storage due to memory shortage. Before brought back to main memory, first returns to the suspend ready, then it returns to the ready state.

Slide 14

Slide Content:
2.2 STATES + PCB
Suspension removes a process from immediate competition
12
READY SUSPENDED
Able to run logically
Not resident in main memory
Must be restored before dispatch
BLOCKED SUSPENDED
Still waiting for an event
Also not resident in main memory
Event completion may change its state
Suspension supports swapping, memory pressure control, and administrative pause.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains suspended process states. A suspended process is temporarily removed from normal CPU-scheduling competition. In the classical OS model, it may also be moved out of RAM to secondary storage.
Ready Suspended
The process has everything it needs to execute, except that it is suspended.
It is logically ready to use the CPU.
Its memory image may have been moved from RAM to storage.
It cannot be selected by the short-term CPU scheduler yet.
It must be resumed–and, if swapped out, restored to RAM–before entering the regular Ready state.

Ready Suspended → restore/resume → Ready → dispatch → Running
Blocked Suspended
The process is suspended and waiting for an event, such as I/O completion.
It cannot run because the required event has not occurred.
It may also be outside main memory.
If the event completes while it remains suspended, it becomes Ready Suspended, not immediately Ready.

Slide 15

Slide Content:
Simple example
Suppose Process A is waiting for a disk operation:
- Process A is Blocked.
- The OS experiences memory pressure and swaps A out → Blocked Suspended.
- The disk operation finishes → Ready Suspended.
- The OS restores A to memory → Ready.
- The scheduler eventually dispatches it → Running. **Speaker Notes (Under Slide):**

One clarification for teaching: "suspended" does not always mean the process is physically outside RAM in every modern OS. A process may also be suspended by an administrator or debugger while some pages remain resident. The slide represents the traditional textbook model that connects suspension with swapping.

Slide 16

Slide Content:
2.2 STATES + PCB
The PCB is the kernel's record for one process
13
IDENTITY
PID and ownership
CPU CONTEXT
PC and registers
STATE
ready, running, waiting
SCHEDULING
priority and queue links
MEMORY + ACCOUNTING
page-table information; CPU time and limits
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

The PCB–Process Control Block–is the kernel's data record for one process.
A process has its program and memory, but the kernel needs its own protected record to manage that process. The PCB is stored in kernel memory, where the user process cannot directly modify it.

Slide 17

Slide Content:
2.2 STATES + PCB
Registers and the program counter make execution resumable
14
PROGRAM COUNTER
• Address of the next instruction
• Changes as instructions execute
• Saved when the process stops running
CPU REGISTERS
• General-purpose values
• Stack pointer and status flags
• Restored before execution resumes

Without saved CPU state, a preempted process could not continue correctly.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Program counter
The Program Counter (PC) is a special CPU register that identifies where execution should continue.
It normally contains the address of the next instruction.
It changes as instructions execute.
A jump, loop, function call, or return can change it to a different address.
When a process stops running, the kernel saves its PC value.
CPU registers
Registers contain the process's current working values, including:
Numbers being calculated
Memory addresses
Function arguments and return values
Stack pointer
Status flags
During a context switch
When Process A stops running:
CPU registers → saved in Process A's PCB
Program counter → saved in Process A's PCB
Before Process A resumes:
Process A's PCB → restore registers
Process A's PCB → restore program counter
CPU continues Process A

Slide 18

Slide Content:
Explain:
The program counter is the timestamp, while the other registers preserve the unfinished details of what the process was doing at that moment. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 19

Slide Content:
2.2 STATES + PCB
Process queues reflect what each process is waiting for
15
Queue
Contains
Leaves when
Job
newly submitted work
admitted
Ready
runnable processes
dispatched
Device
processes blocked for I/O
I/O completes
Event
processes awaiting a condition
event occurs
A process can move among queues many times during its lifetime.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 20

Slide Content:
SECTION 2.3
Process Operations
Creation builds a new execution context; termination releases it in a controlled way
2.3 / 11
16
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 21

Slide Content:
2.3 OPERATIONS
Process creation establishes identity, state, and inheritance
17
Create
allocate PID + PCB
Initialize
address space + state
Inherit
files + attributes
Enqueue
place child in ready queue
The exact inheritance rules depend on the operating system and creation API.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Create: Assigns a unique PID and creates a Process Control Block (PCB).
Initialize: Sets up the process's address space, initial state, and CPU context.
Inherit: Gives the child selected resources from its parent, such as open files, permissions, and environment settings.
Enqueue: Places the new process in the ready queue, where it waits for CPU time.
The exact inherited resources depend on the operating system and the process-creation method, such as Unix fork() or Windows CreateProcess().

Slide 22

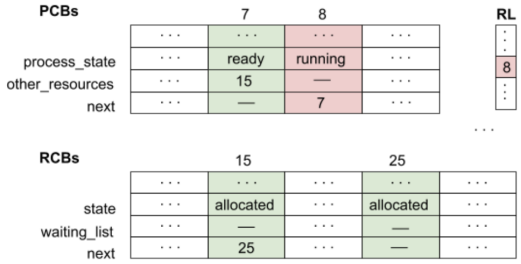
Slide Content:
2.3 OPERATIONS
fork() copies a process; exec() replaces its program
18
fork()
Creates a child process
Child begins from the same logical point
Returns different values to parent and child
exec()
Loads a new program into the caller
Replaces code, data, heap, and stack
PID usually remains unchanged
A common Unix pattern is fork() → child exec().
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 23

Slide Content:
(Visual Slide)

- Two processes (7 and 8) are currently on RL. Process 8 is at the head of the list and is running. The next field of process 8 points to process 7, which is ready.
- Process 7 is currently holding two resources (15 and 25). The other_resources field of process 7 points to resource 15, which in turn points to the next resource, 25.



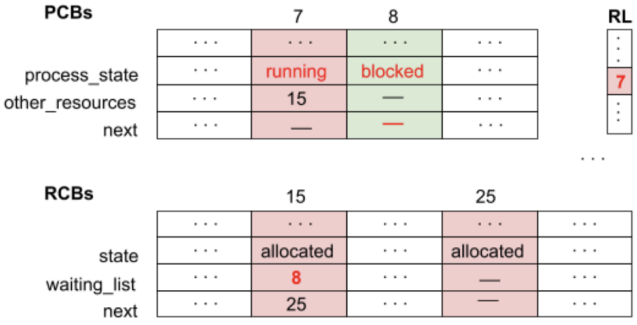
(a) Show all changes to the data structures after process 8 requests resource 15.

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 24

Slide Content:
Solution

- Process 8 is blocked on resource 15. Process 7 is running and is the only process on RL.



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 25

Slide Content:
EXERCISE
Consider a single-CPU system with the following rules:
- The Ready List (RL) contains three priority queues: priority 0, 1, and 2, where 2 is the highest priority.
- Initially, only Process 0 (PCB 0) exists. It has priority 2 and is running.
- create(n) creates a child process with priority n.
- New processes receive the next available PCB number: 1, 2, 3, and so forth.
- A newly created process is placed at the end of its corresponding priority queue.
- Creating a process does not interrupt the currently running process.

- timeout() moves the running process from the head to the end of its current priority queue. The process now at the head of the highest-priority nonempty queue begins running.
The following operations are executed by whichever process is running at that moment:
- create(1)
- create(2)
- timeout()
- create(0)
- timeout()
- create(2)
Question:
At the end of this sequence, how many direct child processes does each process have? Show which process executes each create() operation. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 26

Slide Content:
Solution
TABLE

Operation — Running process — Result
create(1) — P0 — P0 creates P1
create(2) — P0 — P0 creates P2
timeout() — P0 → P2 — P2 begins running
create(0) — P2 — P2 creates P3
timeout() — P2 → P0 — P0 begins running
create(2) — P0 — P0 creates P4
Final result:
P0: 3 children – P1, P2, and P4
P2: 1 child – P3
P1, P3, and P4: 0 children **Speaker Notes (Under Slide):**

2.5.2: Effect of operations on processes and resources.

Slide 27

Slide Content:
2.3 PROCESS OPERATIONS
C example: fork creates a child that can call exec
19
SETUP + FORK
#include ¡stdio.h¿
#include ¡stdlib.h¿
#include ¡unistd.h¿
#include ¡sys/types.h¿
#include ¡sys/wait.h¿
int main(void) {
pid_t pid = fork();
CHECK THE RETURN VALUE
if (pid ¡ 0) {
perror("fork");
return EXIT_FAILURE;
}
if (pid == 0) {
printf("Child PID=%ld\n",
(long)getpid());
Part 1 of 2 • After fork(), parent and child continue from the next statement.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide shows the first part of creating a process in Unix/Linux using fork().
Compile the complete two-slide example with: gcc fork_exec_wait.c -o demo.
Sources

- https://man7.org/linux/man-pages/man2/fork.2.html
/Sources

pid ¡ 0 means that fork() failed to create a child process.
pid == 0 does not mean the child's actual PID is zero. Its actual PID might be 101; zero only tells the program, "You are executing inside the child process."
pid ¿ 0: This code is executing in the parent; pid contains the child's PID. pid ¿ 0 does not mean the parent's PID is positive. It means:
"You are in the parent process, and this positive value identifies the child that was just created."
getpid() // Current process's own PID
getppid() // Current process's parent PID

Slide 28

Slide Content:
2.3 PROCESS OPERATIONS
C example: exec replaces the child; waitpid collects its status
20
CHILD: EXEC
char *args[] = {"ls", "-l", NULL};
execvp(args[0], args);
/* Reached only if exec fails */
perror("execvp");
_exit(127);
}
PARENT: WAIT
int status;
if (waitpid(pid, &status, 0) == -1) {
perror("waitpid");
return EXIT_FAILURE;
}
if (WIFEXITED(status))
printf("Exit status=%d\n",
WEXITSTATUS(status));
return EXIT_SUCCESS;
}
Part 2 of 2 • Test WIFEXITED before using WEXITSTATUS.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This is the parent's code. It waits for the child to finish and then checks how the child ended.
Ask: Why use _exit() after exec failure in the child? It avoids flushing copied stdio buffers inherited from the parent.
Sources

- https://man7.org/linux/man-pages/man3/exec.3.html
- https://man7.org/linux/man-pages/man2/waitpid.2.html
/Sources

The sequence is:
The child calls execvp() to run ls -l.
If successful, execvp() never returns.
If it fails, perror() displays the error.
_exit(127) immediately terminates the child.
The parent's waitpid() receives the status and can obtain 127 using WEXITSTATUS(status).
127 is conventionally used to indicate that a requested command could not be executed or was not found.
execvp() replaces the child's current program code and memory image with the ls program. It does not create another

process, and the child keeps the same PID.
If execvp() succeeds, it never returns. Therefore, these lines run only if it fails:
perror("execvp"); _exit(127);
if (waitpid(pid, &status, 0) == -1)
means: "Wait for the child whose PID is stored in pid, and place information about its termination in status."
if (WIFEXITED(status))
checks whether the child terminated normally, such as by calling exit() or _exit() or by returning from main().
If it ended normally: WEXITSTATUS(status)
extracts the child's actual exit code.

Slide 29

Slide Content:
2.3 PROCESS OPERATIONS
The same code creates two execution paths, then synchronizes them
21
fork()
returns twice
Child: pid = 0
execvp("ls", ...)
Parent: pid ¿ 0
waitpid(pid, ...)
Rejoin
parent collects status
exec succeeds → it never returns
exec fails → child reports the error and exits
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Rejoin: Parent resumes – collects child's termination status
This slide summarizes the complete fork()-exec()-waitpid() sequence:
fork() creates the child.Afterward, the parent and child are separate processes that continue from the next statement.
Child path: pid == 0The child calls execvp() and replaces its original program image with ls.
Parent path: pid ¿ 0The parent calls waitpid() and waits for that specific child to terminate.
Child terminates.The operating system records its termination information.
waitpid() returns.The parent collects the child's status and continues.

Slide 30

Slide Content:
2.3 OPERATIONS
A shell uses fork(), exec(), exit(), and wait() as one workflow
22
1-2
Shell reads a command → Shell calls fork()
3
Child calls exec()
4
Program runs
5-6
Child calls exit() → Parent wait() collects status
A context is preserved before another execution stream takes over.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains what happens when a user enters a command into a Unix/Linux shell.

Slide 31

Slide Content:
=¿
The workflow is:
Shell reads the command.
Shell calls fork(), creating a child process.
Two paths now exist:
- The child calls exec() to replace its program image with ls.
- The parent shell calls wait() and pauses.
The child executes ls -l.
When ls finishes, the child terminates and provides an exit status.
wait() returns, the shell collects the status, and displays the next command prompt.
Shell
Child → exec(ls) → run ls → terminate
Parent shell → wait() → resume **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 32

Slide Content:
2.3 OPERATIONS
exit() ends execution; wait() completes parent-side cleanup
23
exit()
Terminates the calling process
Records an exit status
Releases most resources
wait()
Blocks or polls for child termination
Retrieves the exit status
Allows the kernel to remove the child record
Termination is a protocol between child, kernel, and parent.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains how a child process terminates and how its parent collects the result.
exit(): executed by the terminating process
wait(): executed by the parent

Slide 33

Slide Content:
2.3 OPERATIONS
Zombie and orphan describe different parent-child conditions
24
ZOMBIE
Child has terminated
Exit status has not been collected
Keeps a small process-table entry
ORPHAN
Parent terminates before the child
Child may continue running
The kernel reparents it to init, systemd, or another designated subreaper.
A zombie is dead but uncollected; an orphan may still be alive.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares two different parent-child situations.
Zombie process

A zombie is a child process that has already terminated, but its parent has not yet collected its termination status using wait() or waitpid().
It is no longer executing.
Most of its resources have been released.
The kernel keeps a small process-table record containing its PID and termination status.
It disappears when the parent collects its status.
Zombie: the child is dead, but the parent is alive and has not collected its status.Orphan: the parent is dead, but the child is still alive.

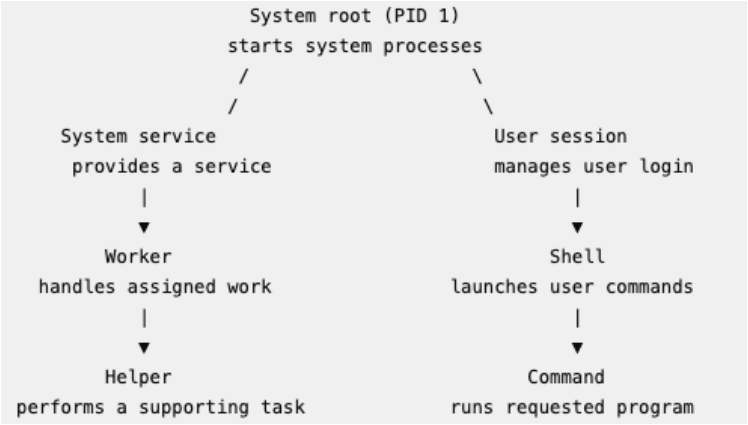
Slide 34

Slide Content:
2.3 OPERATIONS
A process tree exposes ancestry and workload structure
25
System root
starts core services
Service
creates workers
Shell
launches user commands
Worker
may create helpers
Tree structure helps with signaling, accounting, cleanup, and administration.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide means that processes form parent-child relationships. When one process creates another, the creator becomes the parent, and the new process becomes its child.
For example:
System root: PID 1–usually systemd–starts system services.
Service: A server process may create worker processes.
Shell: Creates child processes to execute user commands.
Worker: May create additional helper processes.
Worker process
A worker performs the main tasks assigned by a parent process or service.
Helper process
A helper performs a smaller, specialized supporting task for another process.
A worker might create a helper to

Slide 35

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 36

Slide Content:
SECTION 2.4
Interprocess Communication
Cooperating processes exchange data or coordinate actions through kernel-supported mechanisms
2.4 / 11
26
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 37

Slide Content:
IPC - Interprocess Communication
Separate processes normally have separate protected address spaces. One process cannot directly access another process's variables or memory.
Therefore, the operating system must provide a controlled method for them to:
- Exchange data
- Send notifications
- Coordinate when operations occur
- Synchronize access to shared resources **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 38

Slide Content:
2.4 IPC
Independent and cooperating processes differ in intent
27
INDEPENDENT
Cannot affect another process's computation
No planned information exchange
Strong isolation by default
COOPERATING
Shares data or synchronization
May improve modularity and performance
Requires an IPC protocol
Cooperation creates value–but also synchronization and security risks.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares processes based on whether they intentionally interact.
Cooperating processes
Cooperating processes communicate or coordinate to accomplish related work.
For example:
Web browser process ↔ Network process
They may use:
Pipes , Shared memory , Message queues , Sockets , Signals
A cooperating process requires IPC–Interprocess Communication because separate processes normally have separate protected address spaces. One process cannot directly access another process's variables or memory.
Independent and cooperating processes differ in how they interact

---

Slide 39

Slide Content:
2.4 IPC
Shared memory favors speed; message passing favors isolation
28
SHARED MEMORY
Processes map a common region
Very fast after setup
Requires explicit synchronization
MESSAGE PASSING
Processes send and receive messages
Kernel mediates communication
Natural across protection boundaries
Choose based on data volume, trust, topology, and failure isolation.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares the two main IPC models. Interprocess Communication
Shared memory
The operating system creates a memory region and maps it into the virtual address spaces of multiple processes.
Both processes can directly read and write that region. After setup, communication is fast because they do not need to ask the kernel to transfer every piece of data.
The risk is that both processes might access the same data simultaneously. Therefore, they need synchronization mechanisms such as:
Mutexes
Semaphores
Condition variables
Without synchronization, race conditions or corrupted data can occur.
Message passing
Processes communicate by explicitly sending and receiving messages:
Process A send→ Kernel/IPC channel receive→ Process B
Examples include pipes, message queues, and sockets.
The kernel controls the communication, so processes keep separate address spaces. This provides better isolation and is easier to use across protection boundaries or different computers, but it may be slower because of system calls, data copying, and context switches.
Shared memory is usually faster but harder to coordinate; message passing is usually safer and simpler but may have more overhead.

Slide 40

Slide Content:
2.4 IPC
IPC mechanisms offer different scope and semantics
29
Mechanism
Best fit
Key property
Pipe
related local processes
byte stream
Named pipe
unrelated local processes
filesystem name
Signal
event notification
small asynchronous notice
Socket
local or network peers
bidirectional endpoint

No single IPC mechanism is best for every communication pattern.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares four IPC mechanisms and when each is most suitable.
Pipe
A regular or anonymous pipe usually connects related processes, such as a parent and child.
Named pipe
A named pipe–also called a FIFO–has a pathname in the filesystem.
Signal
A signal sends a small asynchronous notification to a process.
Socket
A socket is a communication endpoint that can connect:
Processes on the same computer using Unix-domain sockets
Processes on different computers using network sockets

Slide 41

Slide Content:
2.4 IPC
An unnamed pipe connects a writer to a reader
30
Create
kernel allocates pipe
Fork
descriptors are inherited
Write
producer sends bytes
Read
consumer receives bytes
A pipe is a byte stream: message boundaries are not automatically preserved.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains how an unnamed pipe allows two related processes–usually a parent and child–to communicate.
Create: The kernel creates a pipe with two file descriptors:
fd[0]: read end
fd[1]: write end
Fork: The child inherits copies of both pipe descriptors from the parent.
Close unused ends
Write
The producer sends bytes into the pipe:
Read
The consumer retrieves bytes

Slide 42

Slide Content:
2.4 IPC
Signals notify a process that an event occurred
31
COMMON USES
• Interrupt or terminate
• Report an exception
• Notify a child-state change
• Deliver a timer event
IMPORTANT LIMITS
• Carries little information
• Handler runs asynchronously
• Unsafe handler actions can corrupt state

• Delivery rules vary by signal
Signals are notifications, not a general-purpose data channel.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains signals, which are short notifications sent to a process when an event occurs. A signal tells the process what type of event happened, but it generally does not carry substantial data.
Signals are suitable for notifying a process that something happened, but pipes, shared memory, or sockets should be used to transfer substantial data.

Slide 43

Slide Content:
2.4 IPC
Local and network IPC face different failures
32
LOCAL IPC
Same machine
Lower latency
Shared kernel and administrative domain
NETWORK IPC
Different machines possible
Packets may be delayed or lost
Peers can fail independently
Sockets provide a common endpoint abstraction across both scopes.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares communication between processes on the same computer with communication across a network. socket provides the same basic communication interface whether two processes are on the same computer or on different computers.

Slide 44

Slide Content:
SECTION 2.5
Thread Fundamentals
Threads divide execution within a process while sharing its address space and resources
2.5 / 11
33
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 45

Slide Content:
2.5 THREADS
A process owns resources; a thread is a stream of execution
34
PROCESS
Own virtual address space
Own resource container
Strong protection boundary
THREAD
Own PC, registers, and stack
Shares code, data, heap, and open files
Scheduled for execution
Every process starts with at least one thread.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 46

Slide Content:
2.5 THREADS
Multithreading improves responsiveness and resource use
35
RESPONSIVENESS
progress despite blocking
RESOURCE SHARING
common address space
ECONOMY
cheaper than processes
SCALABILITY
use multiple cores
STRUCTURE
separate concurrent activities
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 47

Slide Content:
2.5 THREADS
Each thread has private execution state and shared process state
36
PRIVATE PC
next instruction
PRIVATE REGISTERS
current computation
PRIVATE STACK
calls + locals
SHARED CODE
program instructions
SHARED DATA + SHARED FILES
globals + heap; process descriptors
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

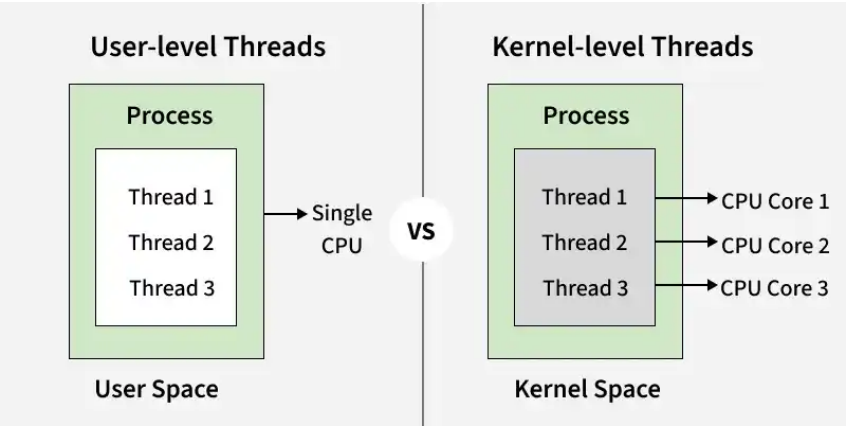
Slide 48

Slide Content:
2.5 THREADS
User-level and kernel-level threads shift control and cost
37
USER-LEVEL
Managed by a runtime library
Fast creation and switching
Kernel may not see each thread
KERNEL-LEVEL
Managed and scheduled by kernel
Can run independently on cores
Operations cost more
The mapping model determines how user threads use kernel scheduling entities.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares who manages the threads.
User-level threads
A thread library inside the application manages them.
The library creates, schedules, and switches between threads.
These operations are fast because they may not require entering the kernel.
The kernel may see only one process or one kernel thread, not every user thread.
In a many-to-one model, one blocking system call can block all user threads.
Multiple user threads cannot run simultaneously on different CPU cores in that model.
Kernel-level threads
The operating-system kernel manages each thread.
The kernel schedules each thread separately.
Different threads from the same process can run simultaneously on different CPU cores.
If one thread blocks, another thread can continue running.
Creating and switching threads costs more because kernel involvement is required.

Slide 49

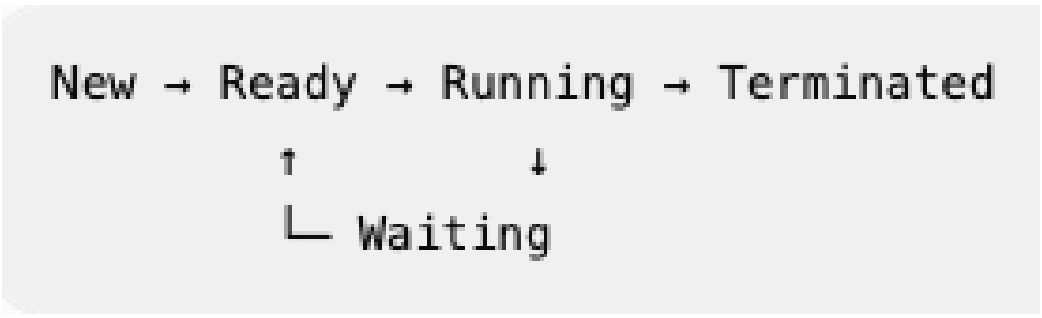
Slide Content:
User-Level Threads: Threads are managed in user space and execute one at a time.
Kernel-Level Threads: Threads are managed by the kernel and can run in parallel on multiple CPUs.



Speaker Notes (Under Slide):
The kernel can see kernel-level threads, but it may not see individual user-level threads.
https://www.geeksforgeeks.org/operating-systems/difference-between-user-level-thread-and-kernel-level-thread/

Slide 50

Slide Content:
2.5 THREADS
A thread moves through states much like a process
38
A Thread Control Block stores the thread's state, PC, registers, stack, and scheduling data.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026



Speaker Notes (Under Slide):
This slide shows that each thread has its own execution state, similar to a process:
New: The thread has been created.
Ready: It can run but is waiting for CPU time.
Running: It is currently executing on a CPU core.
Waiting/Blocked: It is waiting for something, such as I/O, a lock, or another thread.
Terminated: It has completed execution.
The Thread Control Block (TCB) stores the information needed to pause and resume the thread, including its program counter, registers, stack information, state, and scheduling data.

Slide 51

Slide Content:
Exercise - True / False
Mark True or False and justify.
Threads within a process share the same stack.
Kernel-level threads can run in parallel on multicore CPUs.
A virtual CPU is a physical hardware component. **Speaker Notes (Under Slide):**

Answers:
False
True
False

Slide 52

Slide Content:
Exercise - Scenario-Based
A multithreaded application freezes completely when one thread performs a blocking disk read.
Question:
Is the application using user-level threads or kernel-level threads? Why? **Speaker Notes (Under Slide):**

User-level threads, because a blocking system call blocks the entire process.

Slide 53

Slide Content:
A server accepts and processes requests from clients. The server keeps the results of the most recent requests in memory as a cache.
- A new request arrives on average every 25 ms.
- The processing of each request takes 15 ms.
- If the requested result is not in the memory cache, additional 75 ms are needed to access the disk.
- On average, 80% of all requests can be serviced without disk access.
- The creation of a new thread takes 10 ms.

Should the server be implemented as a single-threaded or a multi-threaded process? Justify your answer.
EXERCISE **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 54

Slide Content:
80% of all requests take 15 ms. The remaining 20% take 15 + 75 = 90 ms.
A single-threaded process blocks when the disk is accessed. Thus the average time for each request is 0.8 * 15 + 0.2 * 90 = 30.
The process is unable to keep up with the arrivals and thus an unbounded queue will formA multithreaded process incurs the thread creation time for each request but does not block on disk access. Thus, the average CPU time for each request is 10 + 15 = 25, which is sufficient to keep up with the arrivals.

The multi-threaded approach is the better choice.
Solution **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 55

Slide Content:
SECTION 2.6
Multithreading Models and APIs
Thread mappings determine concurrency, parallelism, and blocking behavior
2.6 / 11
39
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

API stands for Application Programming Interface.
An API is a defined way for one program to request a service from another program or system.
For example, a program may use an operating-system API to:
Create a process
Open or read a file
Allocate memory
Create a thread
Communicate through a network

Slide 56

Slide Content:
2.6 MODELS + APIs
Three mapping models trade simplicity for parallelism
40
Model
Mapping
Consequence
Many-to-one
many user → one kernel
no multicore parallelism
One-to-one
one user → one kernel
parallel but higher overhead
Many-to-many
many user → many kernel
flexible, more complex
Two-level
M:M with bound threads
selective control
Modern general-purpose systems commonly expose one-to-one kernel threads.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 57

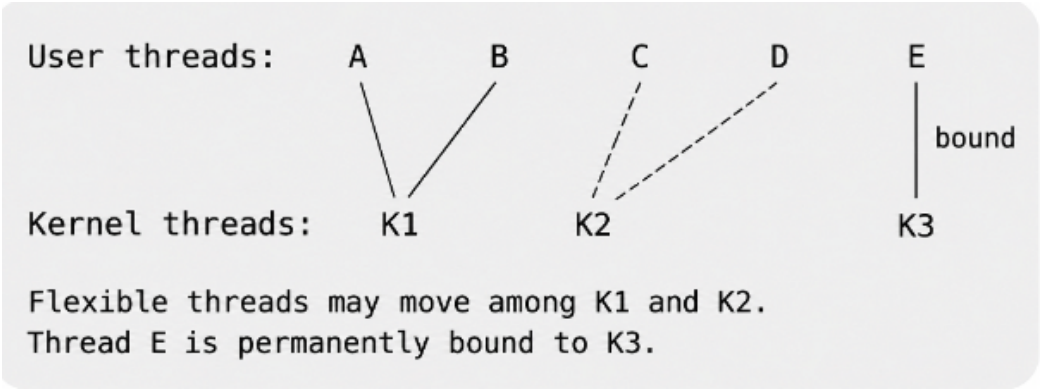
Slide Content:
Thread-mapping model:
- Many-to-one: Many user threads → one kernel thread
- One-to-one: Each user thread → one kernel thread
- Many-to-many: Many user threads → a set of kernel threads
Many-to-one: Many user threads share one kernel thread. It is fast but provides no multicore parallelism. One blocking

system call may block all threads.
One-to-one: Each user thread maps to its own kernel thread. Threads can run on different cores, but kernel management adds overhead.
Many-to-many: Many user threads share a pool of several kernel threads. It supports parallelism with fewer kernel threads, but scheduling is more complex.
Two-level model: A variation of many-to-many. Most user threads can move among kernel threads, but selected user threads can be permanently bound to particular kernel threads. **Speaker Notes (Under Slide):**

Many-to-one: If A requests unavailable data from a disk, K becomes blocked. Because B and C also need K to run, they cannot execute either.
One-to-one: If A requests unavailable data, only K1 becomes blocked. The kernel can still schedule K2 and K3, so B and C may continue running.
Many-to-many: Many user threads share a pool of kernel threads. Each running user thread uses only one kernel thread at a time, but it may use a different kernel thread later. For example, a web server might handle 50,000 client tasks using only 16 kernel threads:
50,000 lightweight user tasks
↓
Runtime scheduler
↓
16 kernel threads
↓
CPU cores

Slide 58

Slide Content:
(Visual Slide)

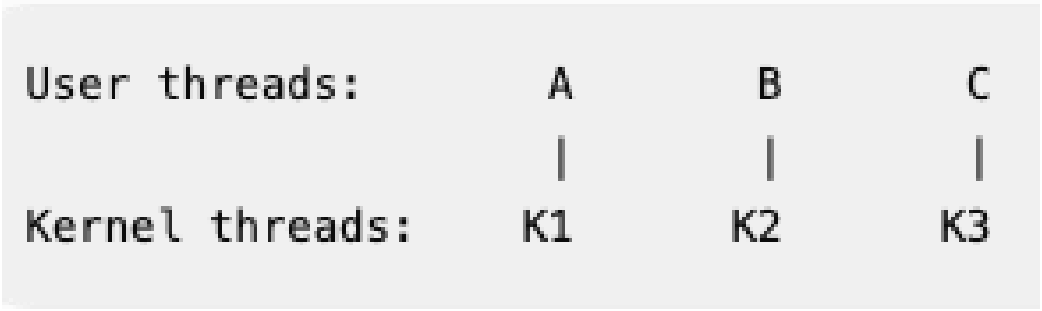


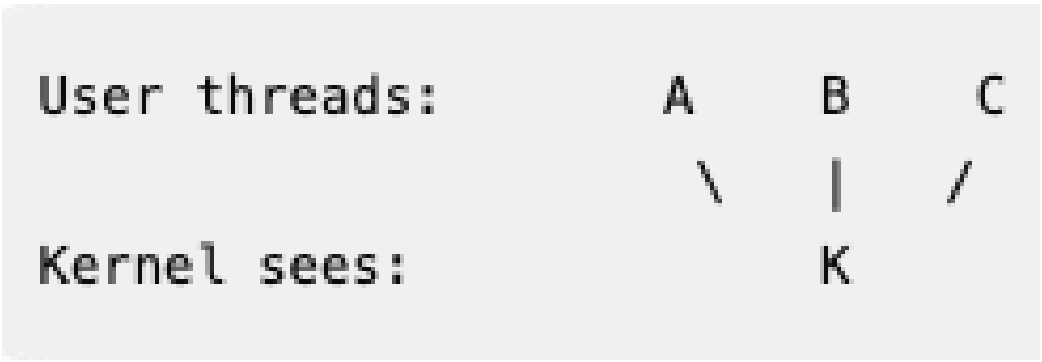

```
User threads:     A       B       C       D       E
                  |               |               |
Kernel threads:   K1              K2              K3

Currently running: A→K1, C→K2, E→K3
Ready and waiting: B, D
```

Later, the mapping can change:

```
User threads:     A       B       C       D       E
                          |               |
Kernel threads:           K1              K2
```





Speaker Notes (Under Slide):
Many-to-one: Many user threads → one kernel thread
One-to-one: Each user thread → one kernel thread
Many-to-many: Many user threads → a set of kernel threads
Two-level model: A variation of many-to-many. Most user threads can move among kernel threads, but selected user threads can be permanently bound to particular kernel threads.

Slide 59

Slide Content:
2.6 MODELS + APIs
Thread APIs Support Direct Threads and Managed Thread Pools
41
PTHREAD_CREATE
start POSIX thread
PTHREAD_JOIN
wait for POSIX thread
JAVA THREAD
Thread object with a Runnable task
JAVA EXECUTOR
ExecutorService manages a thread pool
PYTHON THREADING + ThreadPoolExecutor
Thread object; pool interface
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares threading interfaces in different programming environments
POSIX Threads in C
pthread_create() creates and starts a new thread.
pthread_join() makes the calling thread wait until another thread finishes.
Java
Thread represents a thread.
Runnable defines the task that the thread executes.
ExecutorService manages a pool of worker threads and assigns submitted tasks to them.
Python
threading.Thread creates a thread object.
ThreadPoolExecutor manages a pool and accepts submitted tasks.

Slide 60

Slide Content:
2.6 LINUX CONTEXT
Linux represents processes and threads with a common task abstraction
42
TASK MODEL
• Schedulable entities are represented by task structures
• A thread group shares selected resources
• Each task has its own execution context
clone() / clone3()
• Flags choose what the child shares
• CLONE_VM: address space
• CLONE_FILES: descriptor table
• CLONE_THREAD: thread group
Processes and threads differ mainly in which resources their tasks share.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains that Linux internally treats both processes and threads as "tasks."
The difference is mainly which resources tasks share.
Separate processes usually have separate address spaces and file-descriptor tables.
Threads in the same process share memory and other resources, while retaining their own CPU state and stack.
The clone() or clone3() system call creates a new task. Flags determine what the new task shares with its creator:
CLONE_VM: shares the same virtual address space.
CLONE_FILES: shares the file-descriptor table.
CLONE_THREAD: places the new task in the same thread group.
For example:
Few or no sharing flags → the new task behaves more like a separate process.
Thread-related sharing flags → the new task behaves like a thread in the same process.
The key point is:
Linux does not implement processes and threads as completely different objects. Both are tasks; their resource-sharing configuration determines how they behave.

Slide 61

Slide Content:
2.6 POSIX THREADS
Pthreads example: define a worker and pass one argument
43
HEADERS + WORKER
#include ¡stdio.h¿
#include ¡stdlib.h¿
#include ¡string.h¿
#include ¡pthread.h¿
void *worker_thread(void *arg) {
int id = *(int *)arg;
WORKER BODY
printf("Worker %d; pthread=%lu\n",
id,
(unsigned long)pthread_self());
return NULL;
}
Part 1 of 2 • pthread_self() identifies the pthread; it is not necessarily the kernel TID.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide defines the function that a new POSIX thread will execute:
POSIX: Portable Operating System Interface.

void *arg allows the thread function to receive a pointer to any type of data.
int id = *(int *)arg; converts arg to an int *, dereferences it, and copies the integer value into id.
printf() displays the worker number and its pthread identifier.
pthread_self() returns the calling thread's pthread identifier.
return NULL; terminates the worker thread without returning a result.

Slide 62

Slide Content:
2.6 POSIX THREADS
Pthreads example: create the worker, then join it
44
CREATE

```
int main(void) {
pthread_t thread;
int id = 1;
int rc = pthread_create(
&thread, NULL, worker_thread, &id);
if (rc != 0) {
fprintf(stderr, "pthread_create: %s\n",
strerror(rc));
return EXIT_FAILURE;
}
```

JOIN

```
rc = pthread_join(thread, NULL);
if (rc != 0) {
fprintf(stderr, "pthread_join: %s\n",
strerror(rc));
return EXIT_FAILURE;
}
puts("Worker completed.");
return EXIT_SUCCESS;
}
```

Compile with: gcc pthread_demo.c -pthread -o pthread_demo
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide shows how the main thread creates a worker thread and waits for it to finish.
pthread_t thread;
int id = 1;
thread stores the pthread identifier.
id is the value passed to the worker.
int rc = pthread_create(&thread, NULL, worker_thread, &id);
The four arguments mean:
&thread – location for storing the new thread's identifier
NULL – use default thread attributes
worker_thread – function the new thread will execute
&id – argument passed to that function
After successful creation, the main and worker threads may run concurrently.
rc = pthread_join(thread, NULL);
This makes the main thread wait until the specified worker terminates.
thread identifies which worker to wait for.
NULL means the main thread does not need the worker's return value.
If successful, it returns 0.
If it fails, it returns an error number stored in rc.
Only after the worker finishes does this execute:
puts("Worker completed.");
The overall sequence is:
Main creates worker → both may execute → main calls pthread_join() and waits → worker finishes → main continues

The compilation command is correct:
gcc pthread_demo.c -pthread -o pthread_demo

Slide 63

Slide Content:
2.6 MODELS + APIs
Python threads: concurrency is not always CPU parallelism
45
I/O-BOUND WORK
Threads overlap waits well
Useful for files, sockets, and services
Simple shared-state communication
CPU-BOUND WORK
Global Interpreter Lock (GIL) limits parallel bytecode
Multiprocessing can use multiple cores
Native libraries may release the lock
Choose the concurrency mechanism from the workload, not the syntax.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains that concurrency and parallelism are not the same.
Concurrency means multiple tasks make progress during the same period.
Parallelism means multiple tasks execute at exactly the same time, usually on different CPU cores.
Threads are effective for Concurrency
Processes are effective for parallelism
Python threads are usually effective for I/O-bound work; processes are usually better for CPU-bound Python work.

Slide 64

Slide Content:
2.6 MODELS + APIs
Thread pools bound concurrency and reuse workers
46
WHY POOL THREADS
• Avoid repeated creation cost
• Limit resource consumption
• Queue excess tasks
• Centralize lifecycle management
DESIGN QUESTIONS
• Pool size
• Queue capacity
• Rejection or backpressure policy
• Task cancellation and failure
A pool separates task submission from worker-thread management.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

A thread pool is a fixed or controlled group of reusable worker threads.
Instead of creating a new thread for every task:
The program submits tasks to a queue.
An available worker thread removes a task.
The worker executes it.
After completion, the worker returns to the pool and accepts another task.
Tasks → Waiting queue → Worker threads → Completed tasks
Why use a thread pool?
Avoid repeated creation cost: Creating and destroying threads for every task is expensive.
Limit resource consumption: The pool prevents the program from creating too many threads.
Queue excess tasks: Tasks wait when every worker is busy.
Centralize management: One component controls thread creation, shutdown, cancellation, and errors.

Slide 65

Slide Content:
2.6 MODELS + APIs
Multicore programming introduces new correctness and performance challenges
47
DECOMPOSITION
find parallel tasks
BALANCE
distribute useful work
DEPENDENCIES
coordinate ordering
DATA
partition and protect state
TESTING
reproduce timing bugs
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Parallel speedup requires enough independent work with limited synchronization overhead. **Speaker Notes (Under Slide):**
This slide presents five challenges when writing programs that use multiple CPU cores:
Decomposition: Divide the program into tasks that can run simultaneously. Some operations cannot be parallelized because they depend on earlier results.
Balance: Distribute work evenly. If one core receives most of the work, other cores sit idle and provide little benefit.
Dependencies: Preserve the correct execution order. A task must wait when it needs data produced by another task.
Data: Decide what data each task owns or shares. Shared data requires synchronization to prevent race conditions.
Testing: Multicore bugs depend on timing and thread interleavings. A race condition may appear during one execution and disappear during the next.
Adding more cores does not automatically make a program faster. The work must be divisible, balanced, correctly synchronized, and carefully tested.

Slide 66

Slide Content:
ACTIVE LEARNING
Concept check: many-to-one threads on an 8-core CPU
48
QUESTION
If many user threads map to one kernel thread, how many CPU cores can execute those user threads simultaneously?
DISCUSS
1. Think individually
2. Compare reasoning with a partner
3. Defend one answer using OS state and cost
Be ready to explain the mechanism–not only the answer.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Answer: One core. The kernel sees only one schedulable kernel thread, so the user threads cannot run in parallel on multiple cores.

Slide 67

Slide Content:
SECTION 2.7
Context Switching
A context switch saves one execution context and restores another
2.7 / 11
49
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

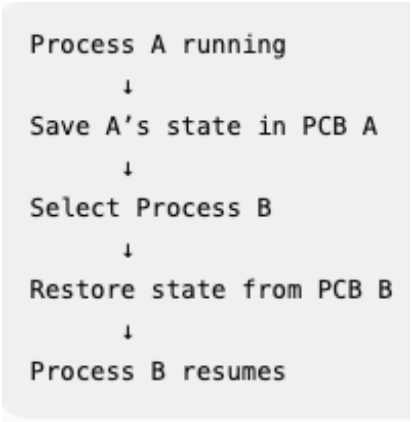
Slide 68

Slide Content:
2.7 CONTEXT SWITCHING
A process context switch follows a controlled save-select-restore sequence
50
1-2
Interrupt or trap enters kernel → Save current CPU state
3
Update current PCB
4
Scheduler selects next process
5-6
Restore selected PCB state → Resume at saved program counter
A context is preserved before another execution stream takes over.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains how the operating system switches the CPU from one process to another.
Enter the kernel: An interrupt, system call, exception, or timer event transfers control to the kernel.
Save the current CPU state: The kernel saves the running process's program counter, registers, stack pointer, and status flags.
Update its PCB: The saved values and new process state–such as Ready or Waiting–are recorded in the current process's PCB.
Select another process: The scheduler chooses a process from the ready queue.
Restore its CPU state: The kernel loads the selected process's saved registers and program counter from its PCB.
Resume execution: The selected process continues from the instruction where it previously stopped.

Slide 69

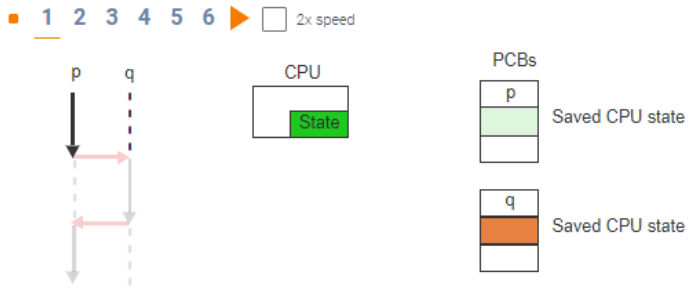
Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 70

Slide Content:
Context switches between processes p and q.

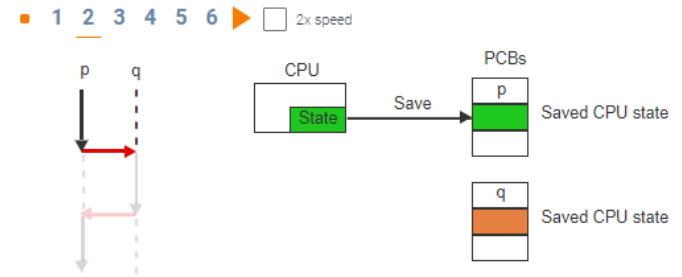


Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 71

Slide Content:
(Visual Slide)

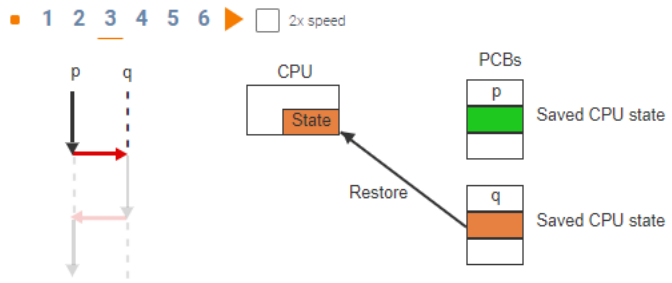


A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 72

Slide Content:
(Visual Slide)

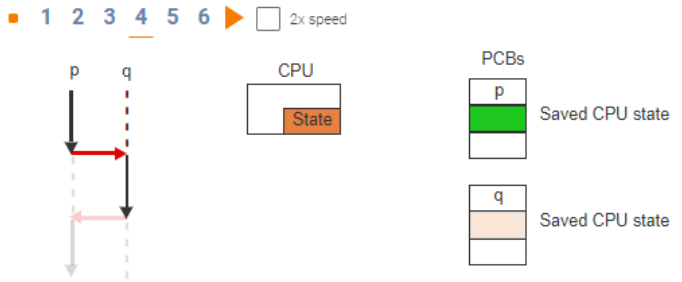


If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 73

Slide Content:
(Visual Slide)

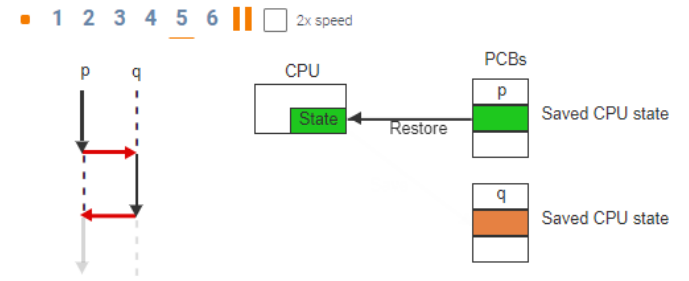


As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 74

Slide Content:
(Visual Slide)

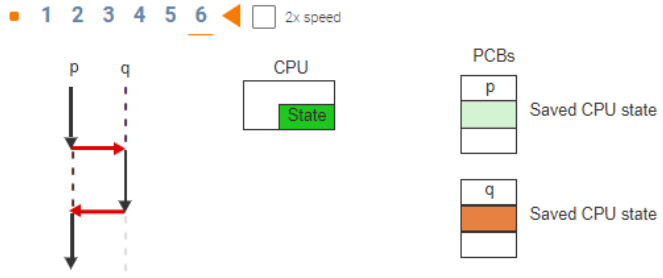


Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 75

Slide Content:
(Visual Slide)



As p continues running, the saved state in the PCB gets out of date.

Speaker Notes (Under Slide):
Captions
Process p is running. The current state of p is in the CPU, while p's PCB contains an out-of-date copy.
A hardware interrupt transfers control to the OS, which saves the current CPU state in p's PCB.
If the OS chooses process q to continue, the current CPU state is overwritten with q's last state from q's PCB.
As q continues running, the saved state in q's PCB gets out of date. q's current state is maintained in the CPU.
Another interrupt causes the current CPU state to be saved in q's PCB. The OS then may restart p by restoring p's saved state in the CPU.
As p continues running, the saved state in the PCB gets out of date.

Slide 76

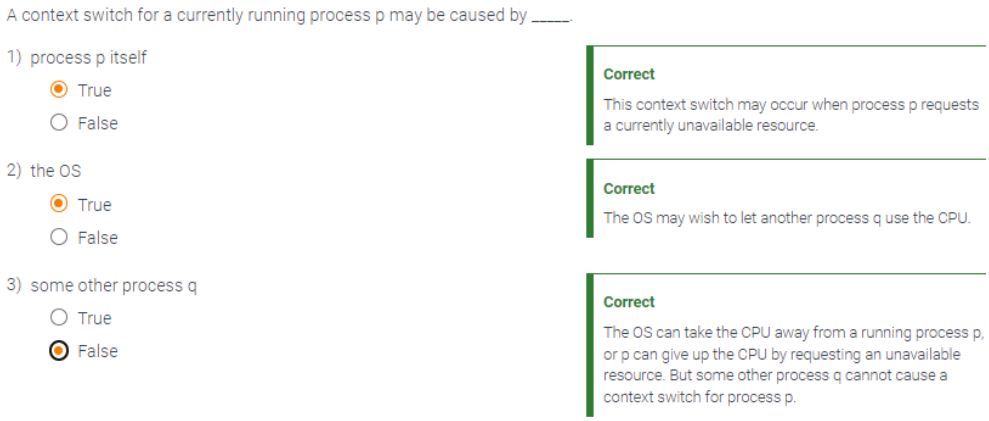
Slide Content:
Exercise:

A context switch for a currently running process p may be caused by _____.
1) process p itself
- T
- F
2) the OS
- T
- F
3) some other process q
- T
- F **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 77

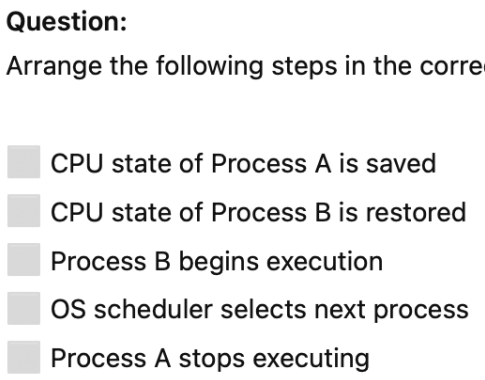
Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 78

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 79

Slide Content:
Answer

1. Process A stops executing
2. CPU state of Process A is saved
3. OS scheduler selects next process
4. CPU state of Process B is restored
5. Process B begins execution

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 80

Slide Content:
2.7 CONTEXT SWITCHING
Thread switches are usually cheaper within the same process
51
PROCESS SWITCH
Changes address-space context
May change page tables and caches
Restores a different PCB
THREAD SWITCH
Changes PC, registers, and stack
Shared address space can remain
Restores a different TCB
Actual cost depends on architecture, cache effects, and kernel design.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 81

Slide Content:
2.7 CONTEXT SWITCHING
Mode switch and context switch answer different questions
52
MODE SWITCH
Privilege level changes
User mode ↔ kernel mode
The same thread may continue
CONTEXT SWITCH
Running execution entity changes
Old state saved; new state restored
May occur after scheduling
A system call causes a mode switch; it does not always cause a context switch.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 82

Slide Content:
2.7 CONTEXT SWITCHING
Context switching is necessary overhead
53
DIRECT COST
• Save and restore state
• Run scheduler and dispatcher
• Update memory-management context
INDIRECT COST
• Cache and TLB disruption
• Pipeline disturbance
• Reduced locality
More switches can improve responsiveness while reducing useful CPU work.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Translation lookaside buffer (TLB): is a memory cache that stores the recent translations of virtual memory addresses to physical memory addresses. It is used to reduce the time taken to access a user memory location.[1] It can be called an address-translation cache. It is a part of the chip's memory-management unit (MMU).
TLB) is a specialized, high-speed hardware cache located inside or near the Memory Management Unit (MMU). Its primary role is to store recently used virtual-to-physical address translations, allowing the CPU to bypass slower page table lookups in main memory. This significantly reduces memory access latency and improves overall CPU performance.
Why it exists: In a virtual memory system, every memory access requires translating a Virtual Page Number (VPN) to a Physical Frame Number (PFN). Without a TLB, this involves two memory accesses – one to fetch the page table entry and another to access the actual data – effectively doubling access time.

Slide 83

Slide Content:
ACTIVE LEARNING
Think-pair-share: which switch is usually cheaper?
54
QUESTION
Process A contains T1 and T2. Process B contains T3. Is switching T1 → T2 usually faster than switching T1 → T3? Why?
DISCUSS
1. Think individually
2. Compare reasoning with a partner
3. Defend one answer using OS state and cost
Be ready to explain the mechanism–not only the answer.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Target: T1 → T2 is usually cheaper because both threads share Process A's address space. Avoid saying "always": architecture, ASIDs/PCIDs, cache state, and kernel implementation affect actual cost.
Switching from to is usually cheaper because they share the same process address space.
Switching from to also requires changing the memory-management context to Process B's address space.
Target: T1 → T2 is usually cheaper because both threads share Process A's address space. Avoid saying "always": architecture, ASIDs/PCIDs, cache state, and kernel implementation affect actual cost.
Switching from _1to _2is usually cheaper because they share the same process address space.
Switching from _1to _3also requires changing the memory-management context to Process B's address space.

Slide 84

Slide Content:
SECTION 2.8

CPU-Scheduling Foundations
Scheduling chooses who runs next; dispatching performs the handoff
2.8 / 11
55
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 85

Slide Content:
2.8 SCHEDULING FOUNDATIONS
Processes alternate between CPU bursts and I/O waits
56
CPU burst
compute
I/O request
block
I/O burst
device works
Wake up
return ready → Next CPU burst: continue
The distribution of CPU bursts strongly influences scheduling performance.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide shows that a process alternates between using the CPU and waiting for I/O.
CPU burst: The process uses the CPU to execute instructions–for example, performing calculations.
I/O request: The process requests an operation, such as reading data from disk. For a blocking request, it enters the waiting state.
I/O burst: The device handles the operation. Meanwhile, the CPU can execute another ready process.
Wake up: When I/O completes, the process becomes ready and waits to be scheduled again.

Slide 86

Slide Content:
2.8 SCHEDULING FOUNDATIONS
CPU-bound and I/O-bound workloads stress different resources
57
CPU-BOUND
Long CPU bursts
Less frequent I/O waiting
Throughput depends on compute capacity
I/O-BOUND
Short CPU bursts
Frequent device or network waits
Response depends on quick dispatch
A healthy mix can keep both CPU and devices busy.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide says CPU-bound processes spend most of their time computing, while I/O-bound processes spend much of their time waiting for input/output.
CPU-bound performance is mainly limited by computation; I/O-bound performance is mainly limited by I/O.
"Throughput depends on compute capacity" means a faster CPU can help CPU-bound workloads complete more work per second.
"Response depends on quick dispatch" means that once an I/O-bound process becomes ready, giving it the CPU promptly lets it perform its short computation and issue its next I/O request. Its overall performance also depends on how quickly the I/O completes.

Slide 87

Slide Content:
TABLE

— CPU-bound — I/O-bound
Main activity — Performing computations — Reading, writing, or waiting for data
CPU bursts — Usually long — Usually short
I/O waits — Less frequent — Frequent
Example — Performing a large numerical calculation — Copying files when disk speed is the bottleneck
Main performance limit — CPU processing capacity — Device or network performance **Speaker Notes (Under Slide):**

This slide says CPU-bound processes spend most of their time computing, while I/O-bound processes spend much of their time waiting for input/output.

Slide 88

Slide Content:
2.8 SCHEDULING FOUNDATIONS
Schedulers operate at different time scales
58
Scheduler
Decision
Frequency
Long-term
admit jobs
low
Medium-term
suspend or resume
occasional
Short-term
select ready process
very high
Dispatcher
perform selected switch
every dispatch
Dispatch latency is the time needed to stop one process and start another running.process and start another.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains the different scheduling roles and how frequently each one operates.
Long-term scheduler: Decides which new jobs to admit for execution. Controls the number and mix of active processes. Acts infrequently because admitting jobs happens less often than switching CPU use.
Medium-term scheduler; Temporarily suspends processes and later resumes them, traditionally by swapping them out of and back into memory. Acts occasionally, such as when memory pressure requires it.
Short-term scheduler; Chooses which process in the ready queue gets the CPU next. Acts very frequently–for example, when a process blocks, terminates, or its time slice expires.
Dispatcher: Gives control of the CPU to the process selected by the short-term scheduler. Acts whenever that selection is put into effect.

Slide 89

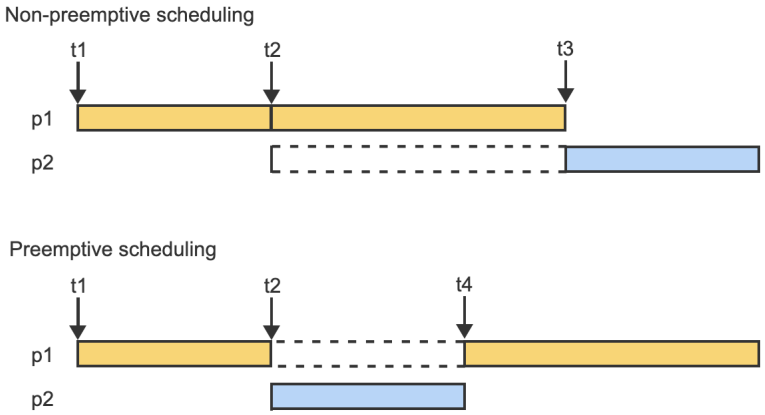
Slide Content:
Preemptive Vs Non-preemptive Scheduling
Non-Preemptive Scheduling
- Running process keeps the CPU until:
- It terminates, or
- It blocks (e.g., I/O request)
- No forced interruption by OS
- Fewer context switches

- Simpler implementation
- May cause poor responsiveness
Preemptive Scheduling
- OS can interrupt a running process
- CPU may be reassigned when:
- A new process enters the ready queue
- A blocked process becomes ready
- A timer interrupt occurs (time slice expires)
- Better responsiveness
- Supports multitasking & time-sharing
- More context switching overhead **Speaker Notes (Under Slide):**

Non-preemptive = process gives up CPU voluntarily.
Preemptive = OS can take CPU away.

Slide 90

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
Captions
Process p1 arrives at time t1 and starts executing.
A second process p2 arrives a time t2.
Under a non-preemptive algorithm, p1 continues while p2 waits.
p2 starts when p1 terminates at time t3.
Under a preemptive algorithm, p1 stops at time t2.
If p2 is chosen to run next, p1 needs to wait until time t4 to continue.

Slide 91

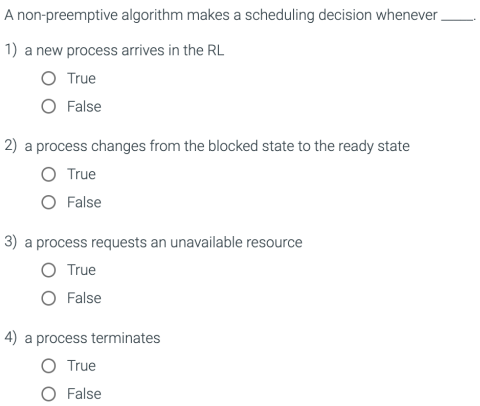
Slide Content:
2.8 SCHEDULING FOUNDATIONS
Preemption improves responsiveness but increases coordination cost
59
NON-PREEMPTIVE
Process keeps CPU until block or exit
Simpler reasoning
A long burst can delay others
PREEMPTIVE
Kernel can reclaim CPU
Supports time sharing and priorities
Shared data needs stronger protection
Timer interrupts make practical preemptive scheduling possible.

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide says preemption can improve response time, but adds CPU-switching overhead and requires careful protection of shared data.
Nonpreemptive scheduling: A process keeps the CPU until it blocks, exits, or voluntarily yields. A long CPU burst can make other processes wait. Execution is easier to reason about because there are fewer possible switching points.
Preemptive scheduling: The OS can take the CPU away from a running process. Other processes can get CPU time before the current process finishes its burst. A process may be paused halfway through updating shared data.
Preemptive: Round Robin and Shortest Remaining Time First.
Nonpreemptive: First-Come, First-Served and nonpreemptive Shortest Job First.

Slide 92

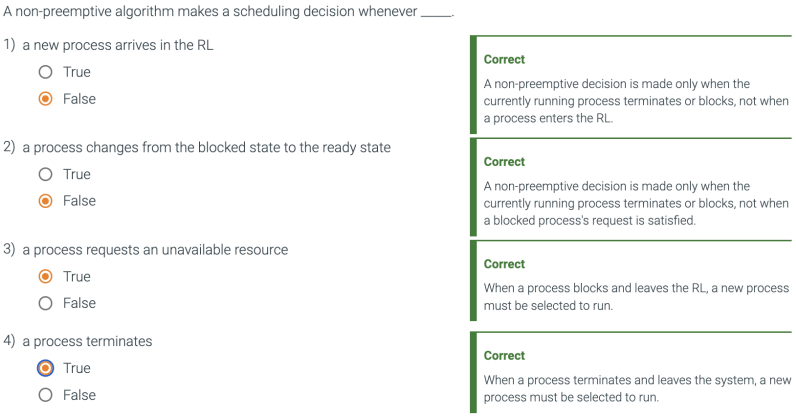
Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 93

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 94

Slide Content:
Exercise 1 (True/False)
In a preemptive priority scheduling system, a lower-priority process can interrupt a higher-priority process.
Aging is used to prevent starvation in priority scheduling.
In real-time systems, missing a deadline may be unacceptable.
A CPU-bound process typically receives higher priority in interactive systems.
Burst time includes both CPU time and I/O time. **Speaker Notes (Under Slide):**

1 False
2 True
3 True
4 False
5 False

Slide 95

Slide Content:
SECTION 2.9
Scheduling Criteria
The best algorithm depends on which performance objective matters most
2.9 / 11
60
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 96

Slide Content:
2.9 CRITERIA
Scheduling quality is measured from several perspectives
61
UTILIZATION
CPU busy percentage
THROUGHPUT
jobs completed per unit time
TURNAROUND
completion - arrival
WAITING
time spent ready
RESPONSE + FAIRNESS
first run - arrival; reasonable access for all
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Creating a Process

Slide 97

Slide Content:
2.9 CRITERIA
Scheduling quality is measured from several perspectives
61
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Criterion — Meaning — Goal
CPU utilization — Percentage of time the CPU is busy. Busy for 90 out of 100 ms = 90%. — Maximize

Throughput — Number of processes completed per unit of time. Completing 20 processes in 10 seconds = 2 processes/second. — Maximize
Turnaround time — Total time from arrival to completion: completion - arrival. — Minimize
Waiting time — Total time spent in the ready queue, waiting for the CPU. — Minimize
Response time — Time until the process first gets the CPU: first start - arrival. — Minimize
Fairness — Giving processes reasonable CPU access and preventing indefinite postponement, called starvation. — Maintain **Speaker Notes (Under Slide):**

This slide explains how we evaluate scheduling: keeping the CPU busy, completing work, reducing delays, and giving processes fair access.

Slide 98

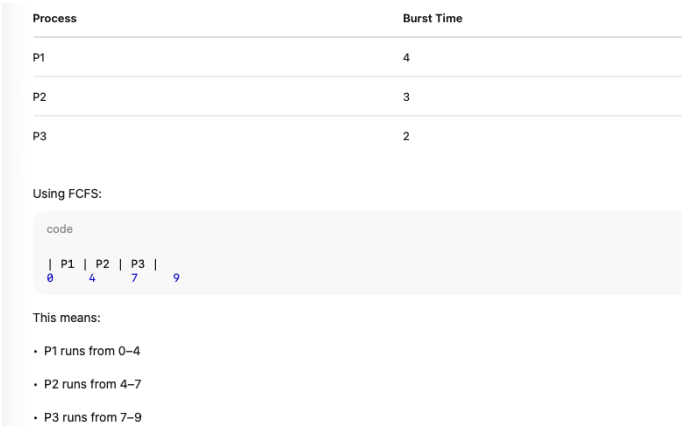
Slide Content:
TABLE

Measurement — Calculation
Response time — 2 - 0 = 2 ms
Waiting time — Initial wait of 2 + later wait of 4 = 6 ms
Turnaround time — 12 - 0 = 12 ms
For a simple example, suppose process A:
Arrives at 0 ms.
Waits, then first runs from 2-5 ms.
Gets preempted and waits from 5-9 ms.
Runs again from 9-12 ms, then finishes.
Its measurements are: **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 99

Slide Content:
What Is a Gantt Chart?
A Gantt chart is a timeline diagram used to show:
Which process is running on the CPU and for how long.



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 100

Slide Content:
(Visual Slide)

Given:

Process	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	1	4	4
P3	2	2	6
P4	3	1	3

👉 Higher number = Higher priority
👉 Preemptive scheduling

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 101

Slide Content:
Step-by-Step Execution
- Time 0: Only P1 has arrived → Runs
- Time 1: P2 arrives (priority 4 ¿ 2)
- Preempts P1
- P2 runs
- Time 2: P3 arrives (priority 6 ¿ 4)
- Preempts P2
- P3 runs
- Time 3: P4 arrives (priority 3 ¡ 6)
- No preemption (P3 still highest)
- Time 4: P3 finishes
- Highest priority in ready queue = P2
- P2 resumes
- Time 7: P2 finishes
- Highest priority = P4
- P4 runs
- Time 8: P4 finishes
- Only P1 remains
- P1 resumes
- Time 15: P1 finishes

```
| P1 | P2 | P3 | P2 | P4 | P1 |
0    1    2    4    7    8    15
```

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 102

Slide Content:
(Visual Slide)

Given:

Process	Arrival	Burst	Priority
P1	0	6	2
P2	1	4	5
P3	2	2	3

Higher number = higher priority
Non-preemptive scheduling

Questions:

1. Draw the Gantt chart.
2. Compute waiting time for each process.
3. Compute average waiting time.

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 103

Slide Content:
Solution Outline
- Execution Order:
- Time 0 → P1 runs (only one available)
- Time 6 → P2 (highest priority)
- Then P3
- Gantt Chart:
- Waiting Times:
- P1 = 0
- P2 = 6 - 1 = 5
- P3 = 10 - 2 = 8
- Average Waiting Time = (0 + 5 + 8) / 3 = 4.33

Gantt Chart:

```
code

| P1 | P2 | P3 |
0    6    10   12
```

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 104

Slide Content:
2.9 CRITERIA
Scheduling criteria: which values should we maximize or minimize?
62
Criterion
Prefer

Why
CPU utilization
higher
avoid idle capacity
Throughput
higher
finish more work
Turnaround
lower
complete jobs sooner
Waiting / Response
lower / lower
reduce ready-queue delay; react quickly to users
Predictability matters too: low variance can be more valuable than a slightly better average.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide says interactive systems prioritize quick responses, while batch systems prioritize completing jobs efficiently. Interactive systems respond to users as they work; batch systems execute submitted jobs with little or no user interaction while those jobs run.
predictability:: consistent response times can provide a better experience than usually fast responses with occasional long delays. "Low variance" means those response times vary less.

Slide 105

Slide Content:
SECTION 2.10
Scheduling Algorithms
Algorithm behavior becomes clear when the same workload is drawn as a Gantt chart
2.10 / 11
63
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 106

Slide Content:
What Is an Arbitration Rule?
- An arbitration rule is used:
- When two or more processes have the same priority.
- It decides which one runs first.
- Examples:
- FIFO (First-Come, First-Served)
- SJF (Shortest Job First)
- Lowest PID
- Round Robin
- These rules break ties.
- They do not change priority values.
- Breaks ties between equal priorities **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 107

Slide Content:
Scheduling of Batch Processes
What Are Batch Processes?
- Batch processes are jobs that:

- Run without user interaction
- Are submitted in advance
- Execute sequentially by the operating system
- Focus on throughput, not response time
- Examples:
- Payroll processing
- Scientific simulations
- Data analysis jobs **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 108

Slide Content:
Goals of Batch Scheduling
- The scheduler aims to:
- Maximize CPU utilization
- Maximize throughput (jobs completed per unit time)
- Minimize average waiting time
- Minimize turnaround time
- (User responsiveness is not a priority.) **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 109

Slide Content:
2.10 ALGORITHMS
Use one workload to compare scheduling decisions
64
Process
Arrival
CPU burst
P1
0
8
P2
1
4
P3
2
2
P4
3
1
Assume lower number means higher priority only on the priority example.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 110

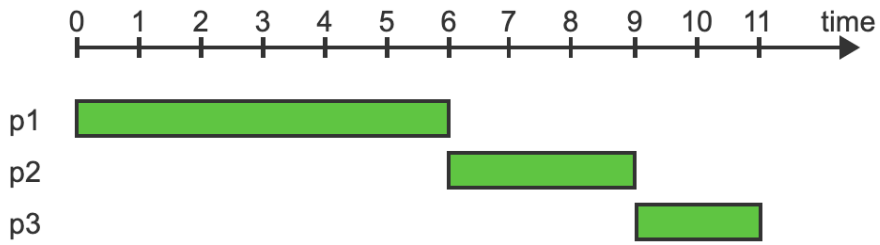
Slide Content:
Common Scheduling Algorithms for Batch Systems

- 1 First-Come, First-Served (FCFS)
- Processes run in arrival order.
- Simple but may cause long waiting times.
- FIFO is non-preemptive. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 111

Slide Content:
Execution Under FIFO Scheduling



Speaker Notes (Under Slide):
Captions
Three processes need to be scheduled. p1 arrives at t = 0 and will need 6 units of CPU time. p2 arrives at t = 1 and will need 3 units. p3 arrives at t = 3 and will need 2 units.
p1 arrived first and starts immediately at t = 0.
p2 arrived second and will start as soon as p1 terminates at t = 6.
p3 is the last arriving process and will start when p2 terminates at t = 9.

Slide 112

Slide Content:
2.10 ALGORITHMS
FCFS scheduling: question
First-Come, First-Served (FCFS) runs ready processes in arrival order.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Process — Arrival time — CPU burst
P1 — 0 — 8
P2 — 1 — 4
P3 — 2 — 2
P4 — 3 — 1
Using nonpreemptive FCFS:
1. Draw the Gantt chart.
2. Find each process's waiting,
turnaround, and response times.
3. Calculate the average of
each of these three measures.
Assume one CPU, one CPU burst per process, no I/O, and no context-switch overhead.
FCFS is nonpreemptive: a later arrival does not interrupt the running process.
All times are in time units. **Speaker Notes (Under Slide):**

Ask students to schedule P1 (arrival 0, burst 8), P2 (arrival 1, burst 4), P3 (arrival 2, burst 2), and P4 (arrival 3,

burst 1). FCFS is nonpreemptive. There is a single CPU and each process has one CPU burst; ignore context-switch overhead. Waiting time is total time in the ready queue, turnaround is completion minus arrival, and response is first start minus arrival.

Slide 113

Slide Content:
2.10 ALGORITHMS
FCFS scheduling: answer
The CPU runs P1 → P2 → P3 → P4; each process finishes before the next starts.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
P1
P2
P3
P4
0
8
12
14
15
Gantt chart • not to scale
TABLE

Process — Completion — Waiting — Turnaround — Response
P1 — 8 — 0 — 8 — 0
P2 — 12 — 7 — 11 — 7
P3 — 14 — 10 — 12 — 10
P4 — 15 — 11 — 12 — 11
Average — – — 7 — 10.75 — 7
Waiting = start - arrival • Turnaround = completion - arrival • Response = first start - arrival
P1 keeps the CPU until time 8, even though P2, P3, and P4 arrive earlier. **Speaker Notes (Under Slide):**

P1 starts at time 0 and completes at time 8. P2, P3, and P4 arrive at 1, 2, and 3, and wait in that order. P2 runs from 8 to 12, P3 from 12 to 14, and P4 from 14 to 15. Waiting times are 0, 7, 10, and 11 (average 7). Turnaround times are 8, 11, 12, and 12 (average 10.75). Response times are 0, 7, 10, and 11 (average 7). Waiting equals response for this single-burst, nonpreemptive example. All times are in time units. The displayed Gantt segments are equal-width for readability and explicitly marked not to scale.

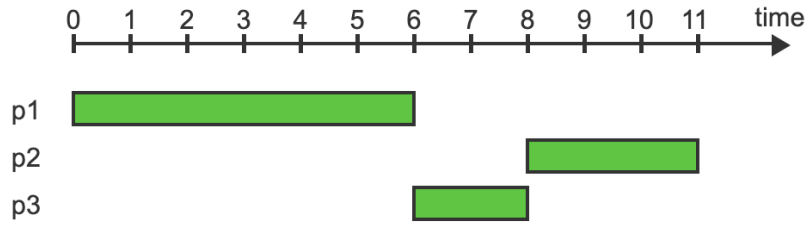
Slide 114

Slide Content:
2 Shortest Job First (SJF)
- Process with smallest CPU burst runs first.
- Minimizes average waiting time.
- Requires burst-time estimation.
- SJF is non-preemptive. **Speaker Notes (Under Slide):**

A more appropriate name for SJF would be "Shortest Total CPU Time First". For historical reasons, the term "Job" has remained in common use.

Slide 115

Slide Content:
Execution under SJF scheduling



Speaker Notes (Under Slide):
Captions ^
Three processes need to be scheduled. p1 arrives at t = 0 and will need 6 units of CPU time. p2 arrives at t = 1 and will need 3units. p3 arrives at t = 3 and will need 2 units.
51 arrived first and starts immediately at t = 0.
p3 has a shorter total CPU time requirement than p2 and will start as soon as p1 terminates at t = 6.
p2 starts when p3 terminates at t = 8.

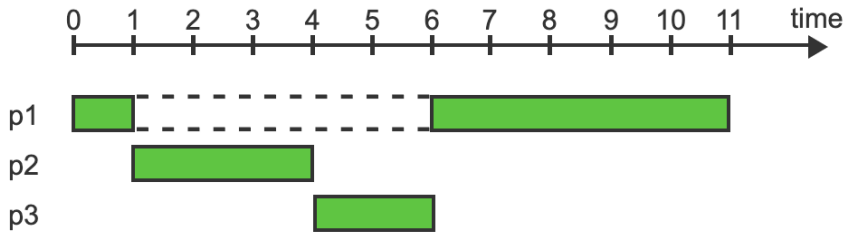
Slide 116

Slide Content:
3 Shortest Remaining Time First (SRTF)
- Preemptive version of SJF.
- Always runs job with least remaining time.
- SRT is the preemptive version of SJF. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 117

Slide Content:
Execution under SRTF scheduling



Speaker Notes (Under Slide):
Captions
Three processes need to be scheduled. p1 arrives at t = 0 and will need 6 units of CPU time. p2 arrives at t = 1 and will need 3 units. p3 arrives at t = 3 and will need 2 units.
p1 arrives first and starts immediately at t = 0.
p2 arrives at t = 1. p2's remaining time is 3 units while p1's remaining time is 5 units. Thus p2 preempts p1 and starts at t = 1.
p3 arrives at t = 3. p3's remaining time is 2 units while p2's remaining time is only 1 unit. Thus p2 continues running until t = 4.

At t = 4, p1's remaining time is 5 units and p3's remaining time is 2 units. Thus p3 starts and runs until t = 6. Finally, p1 restarts at t = 6 and terminates at t = 11.

Slide 118

Slide Content:
What Is Aging?
- Aging is a technique used in priority scheduling to prevent starvation.
- Aging gradually increases the priority of a process the longer it waits in the ready queue.
- It modifies the priority over time to prevent starvation.
- Why Do We Need Aging?
- In priority scheduling:
- High-priority processes run first.
- Low-priority processes may never get CPU time.
- This leads to starvation.
- Aging fixes that problem.
- How Aging Works
- Example:
- Suppose priorities range from 0 (lowest) to 10 (highest).
- A process starts with priority 2.
- For every 5 time units it waits, its priority increases by 1.
- Eventually, it becomes high enough to run. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 119

Slide Content:
2.10 ALGORITHMS
Priority scheduling runs importance first–but needs an anti-starvation policy
71
PRIORITY RULE
Select highest-priority ready process
Can be preemptive or non-preemptive
Priority may be static or dynamic
AGING
Gradually improves waiting-process priority
Ensures long-waiting work eventually runs
Preserves priority differentiation
Priority without aging can postpone low-priority work indefinitely.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide says priority scheduling selects the highest-priority ready process, and aging helps prevent low-priority processes from waiting indefinitely.

Slide 120

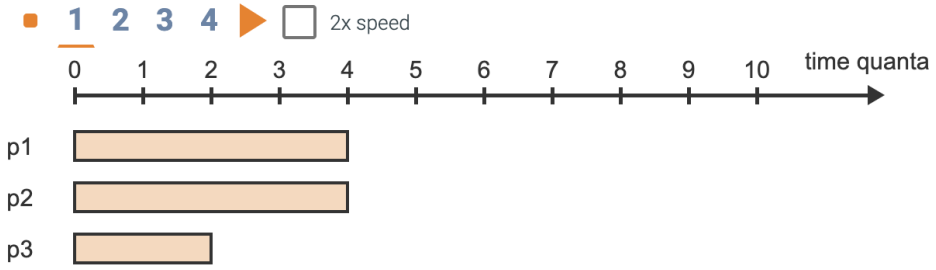
Slide Content:
RR Scheduling Algorithm
Time Quantum (Q)
- A short fixed time (≈ 10-100 ms) a process can use the CPU
Round Robin (RR) Scheduling
- Processes are placed in a single queue
- CPU is given to the process at the front of the queue
How It Works
- Each process runs for Q time units
- If not finished → moved to the end of the queue
- Next process gets the CPU

Key Idea
- Fair CPU sharing using preemptive time-slicing **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 121

Slide Content:
Execution Under RR Scheduling.

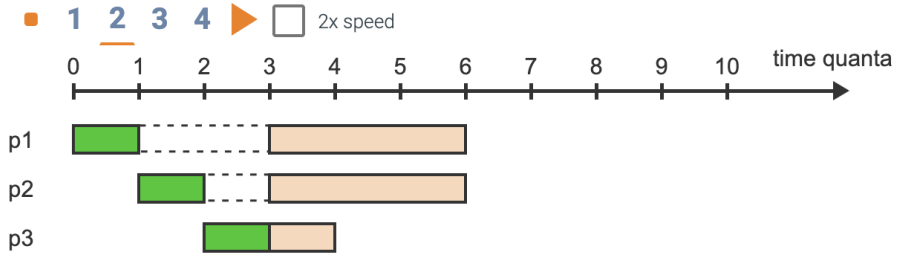


Three processes with total CPU times of 4, 4, and 2 start executing under RR.

Speaker Notes (Under Slide):
Captions
Three processes with total CPU times of 4, 4, and 2 start executing under RR.
p1 executes for 1 quantum, then p2, then p3.
The same cycle repeats: p1, p2, p3 execute for 1 quantum each. p3 terminates.
p1 and p2 then share the CPU using RR until both terminate.

Slide 122

Slide Content:
(Visual Slide)

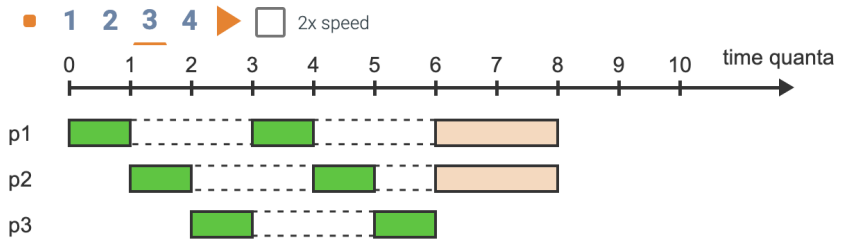


p1 executes for 1 quantum, then p2, then p3.

Speaker Notes (Under Slide):
Captions
Three processes with total CPU times of 4, 4, and 2 start executing under RR.
p1 executes for 1 quantum, then p2, then p3.
The same cycle repeats: p1, p2, p3 execute for 1 quantum each. p3 terminates.
p1 and p2 then share the CPU using RR until both terminate.

Slide 123

Slide Content:
(Visual Slide)

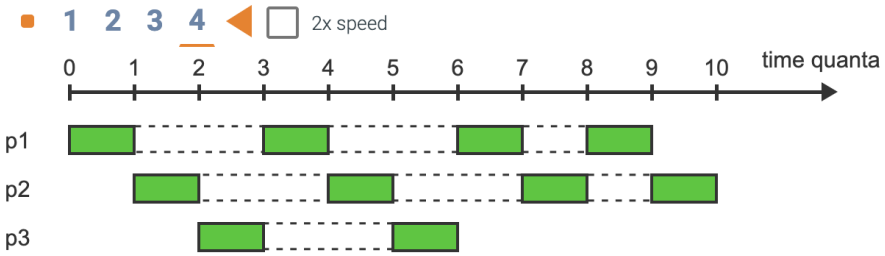


The same cycle repeats: p1, p2, p3 execute for 1 quantum each. p3 terminates.

Speaker Notes (Under Slide):
Captions
Three processes with total CPU times of 4, 4, and 2 start executing under RR.
p1 executes for 1 quantum, then p2, then p3.
The same cycle repeats: p1, p2, p3 execute for 1 quantum each. p3 terminates.
p1 and p2 then share the CPU using RR until both terminate.

Slide 124

Slide Content:
(Visual Slide)



p1 and p2 then share the CPU using RR until both terminate.

Speaker Notes (Under Slide):
Captions
Three processes with total CPU times of 4, 4, and 2 start executing under RR.
p1 executes for 1 quantum, then p2, then p3.
The same cycle repeats: p1, p2, p3 execute for 1 quantum each. p3 terminates.
p1 and p2 then share the CPU using RR until both terminate.

Slide 125

Slide Content:
Exercise:
Tasks:
Draw the Gantt Chart using Round Robin scheduling

Calculate Completion Time (CT) for each process
Calculate Turnaround Time (TAT = CT - AT)
Calculate Waiting Time (WT = TAT - BT)
Compute Average Waiting Time
What happens if Q = 2 ms vs Q = 8 ms?
Which gives better responsiveness and why?

Time Quantum **Q = 4 ms**

Process	Arrival Time	Burst Time
P1	0	10
P2	1	5
P3	2	8

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 126

Slide Content:
Gantt Chart
- 0-4: P1
- 4-8: P2
- 8-12: P3
- 12-16: P1
- 16-17: P2
- 17-21: P3
- 21-23: P1

Completion Time (CT)
- P1 = 23
- P2 = 17
- P3 = 21

Turnaround Time
- TAT = CT - AT
- P1 = 23 - 0 = 23
- P2 = 17 - 1 = 16
- P3 = 21 - 2 = 19

Waiting Time
- WT = TAT - BT
- P1 = 23 - 10 = 13
- P2 = 16 - 5 = 11
- P3 = 19 - 8 = 11

Average Waiting Time
- (13 + 11 + 11)/3 = 35/3 = 11.67 ms

Average Turnaround Time
- 23 + 16 + 19/3= 58/3 = 19.33 ms **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 127

Slide Content:
2.10 ALGORITHMS
The time quantum controls the balance between responsiveness and overhead
74

QUANTUM TOO SMALL
Fast rotation
Many context switches
Lower useful CPU fraction
QUANTUM TOO LARGE
Lower switch overhead
Slower interactive response
Approaches FCFS behavior
Choose q well above context-switch time but short enough for the response target.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Abstraction in an operating system (OS) is the process of hiding raw, complex hardware details and presenting a clean, simplified model to users and programs. It turns physical components–like transistors, disk sectors, and memory chips–into organized, easy-to-use concepts like files, processes, and windows

---

Slide 128

Slide Content:
Impact Of Quantum Size.
n processes are time-sharing the CPU. The average total CPU time is T. The time quantum is Q. The context switching overhead is S. Determine the fraction of CPU time wasted on context switching for the following cases:
- (a)T ¡ Q
- (b)T ¿¿ Q **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 129

Slide Content:
Answer
n processes are time-sharing the CPU. The average total CPU time is T. The time quantum is Q. The context switching overhead is S. Determine the fraction of CPU time wasted on context switching for the following cases:
- (a)T ¡ Q
- When T ¡ Q, the quantum never expires. The execution repeats the timing sequence: T, S, T, S, ... Thus for every T ms of execution, T + S ms of CPU time are needed. The fraction of wasted CPU time is S/(T + S).
- (b)T ¿¿ Q
- When T ¿¿ Q, each execution of T needs many quanta to complete. Each quantum is followed by a context switch. The fraction of wasted CPU time is S/(Q + S). **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 130

Slide Content:
2.10 ALGORITHMS
No basic algorithm dominates every scheduling criterion
75
Gantt charts show order; metrics show performance.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Algorithm — Preemptive? — Main strength — Main drawback
FCFS — No — Simple: processes run in arrival order. — A long job can delay all the short jobs behind it.
SJF — No — Choosing short CPU bursts helps reduce average waiting time. — Requires burst-time estimates; long jobs may starve.
SRTF — Yes — Newly arriving short jobs can run promptly. — More switching; long jobs may starve.
Priority — Either — Gives preference to more important processes. — Low-priority processes may starve without measures such as aging.
Round Robin — Yes — Gives ready processes regular turns, supporting interactive responsiveness. — A small quantum

increases switching overhead; a large quantum increases delays between turns. **Speaker Notes (Under Slide):**

This slide says no single scheduling algorithm is best at everything–the best choice depends on the system's goals.

Slide 131

Slide Content:
ACTIVE LEARNING
Concept check: can SRTF starve a long job?
76
QUESTION
A long CPU-bound job is ready. Short I/O-bound jobs keep arriving. What may happen under SRTF, and how could the OS prevent it?
DISCUSS
1. Think individually
2. Compare reasoning with a partner
3. Defend one answer using OS state and cost
Be ready to explain the mechanism–not only the answer.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide asks whether SRTF can make a long job wait indefinitely and how the OS could prevent that.
Target: repeated short arrivals may indefinitely postpone the long job. Aging, maximum wait bounds, or a multilevel feedback policy can raise its effective priority.
Yes–SRTF can cause starvation if processes with shorter remaining CPU bursts keep becoming ready.

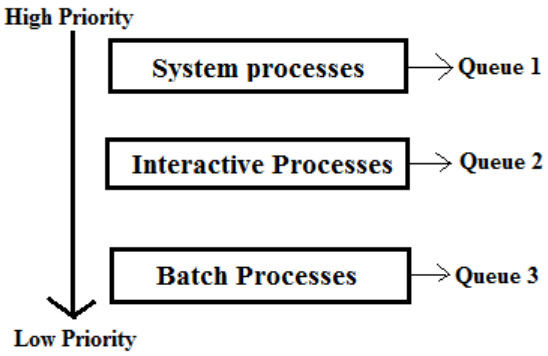
Slide 132

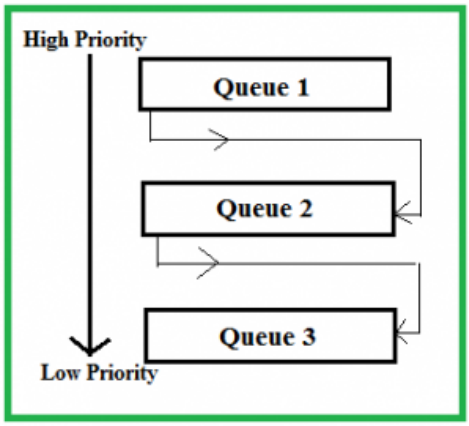
Slide Content:
SECTION 2.11
Advanced Scheduling
Modern schedulers combine queues, feedback, affinity, balancing, and deadline policies
2.11 / 11
77
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 133

Slide Content:
2.11 ADVANCED
Multilevel queues separate classes; feedback queues also move processes
78
MULTILEVEL QUEUE (MLQ)
Fixed assignment to a queue
Each queue may use its own algorithm
Scheduling also occurs among queues
MULTILEVEL FEEDBACK (MLFQ)
Processes move between queues
Behavior changes priority dynamically
Favors short or interactive bursts
Feedback approximates burst prediction using observed behavior.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026

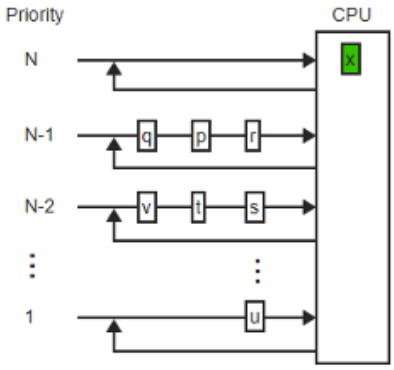


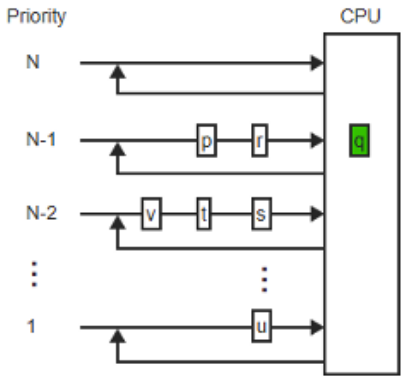


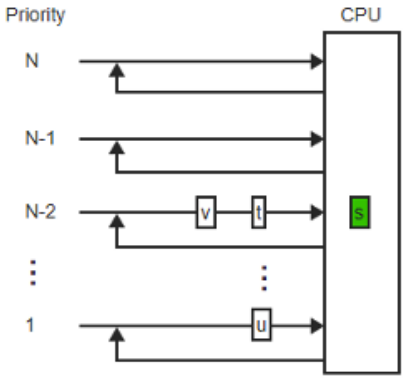
Speaker Notes (Under Slide):
Treat different classes of processes differently.

Slide 134

Slide Content:
MULTILEVEL QUEUE (MLQ)







Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 135

Slide Content:
Process p is in the queue at level N-3, followed by process q at the same level. Queues at levels N through N-2 are empty. The quantum size is 1 time unit.
- 1) If p needs 2 units of CPU time and q needs 1 unit of CPU time, process _____ will terminate first.
- p
- q
- 2) When p starts executing, a new process r with a CPU time requirement of 3 units arrives at level N. The 3 processes will terminate in the order _____.
- r, p, q
- r, q, p
- p, r, q **Speaker Notes (Under Slide):**

Multilevel (ML)

Slide 136

Slide Content:
(Visual Slide)

Process p is in the queue at level N-3, followed by process q at the same level. Queues at levels N through N-2 are empty. The quantum size is 1 time unit.

1) If p needs 2 units of CPU time and q needs 1 unit of CPU time, process _____ will terminate first.
   - ○ p
   - ◉ q

Correct

p starts, executes for 1 unit, and goes back behind q. q needs only 1 time unit and terminates before p.

2) When p starts executing, a new process r with a CPU time requirement of 3 units arrives at level N. The 3 processes will terminate in the order _____.
   - ○ r, p, q
   - ◉ r, q, p
   - ○ p, r, q

Correct

r arrives at a higher level than p and q and thus preempts p, which goes behind q. When r terminates, q starts and terminates within 1 time unit, followed by p.

Speaker Notes (Under Slide):
Multilevel (ML)

Slide 137

Slide Content:
Queue priorities are Q0 ¿ Q1 ¿ Q2; always select from the highest-priority nonempty queue.
Q0 and Q1 use Round Robin, with processes joining the tail of the queue.
Assume no I/O, no aging or priority boosts, and zero context-switch overhead.

Question

A system uses **Multilevel Feedback Queue (MLFQ)** scheduling with 3 levels:

- **Q0**: time quantum = **2 ms**
- **Q1**: time quantum = **4 ms**
- **Q2**: **FCFS**

Rules:

- New processes enter **Q0**
- If a process uses its full quantum and is not finished, it moves to the next lower queue

Process	Arrival Time	Burst Time
P1	0	10
P2	1	4
P3	2	6

Tasks

1. Draw the Gantt chart
2. Show queue movement
3. Find completion time for each process

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 138

Slide Content:
Answer
Step 1: Scheduling
- t = 0 to 2: P1 runs in Q0 for 2 ms, remaining = 8 → moves to Q1
- t = 2 to 4: P2 runs in Q0 for 2 ms, remaining = 2 → moves to Q1
- t = 4 to 6: P3 runs in Q0 for 2 ms, remaining = 4 → moves to Q1
Now Q0 is empty, so move to Q1:
- t = 6 to 10: P1 runs in Q1 for 4 ms, remaining = 4 → moves to Q2
- t = 10 to 12: P2 runs in Q1 for 2 ms and finishes
- t = 12 to 16: P3 runs in Q1 for 4 ms and finishes
Now Q1 is empty, so move to Q2:
- t = 16 to 20: P1 runs in Q2 and finishes
Gantt Chart
- 0-2 P1 — 2-4 P2 — 4-6 P3 — 6-10 P1 — 10-12 P2 — 12-16 P3 — 16-20 P1
Queue Movement

- P1: Q0 → Q1 → Q2
- P2: Q0 → Q1 → Finished
- P3: Q0 → Q1 → Finished

Completion Times

Process	Completion Time
P1	20
P2	12
P3	16

Speaker Notes (Under Slide):
At time 1, P2 does not interrupt P1: Both are in Q0, so P1 continues until its 2 ms quantum expires.
At time 16, P3 finishes exactly at the end of its Q1 quantum: It exits rather than moving to Q2. Demotion applies only if the process still needs CPU time.

Slide 139

Slide Content:
Question: Multilevel Feedback Queue Scheduling
A system uses Multilevel Feedback Queue (MLFQ) scheduling with four queues:
Queue Scheduling algorithm Time quantum
Q0 – highest priority Round Robin 1 ms
Q1 Round Robin 2 ms
Q2 Round Robin 4 ms
Q3 – lowest priority FCFS No fixed quantum
Scheduling rules
- New processes enter Q0.
- The CPU always selects a process from the highest-priority nonempty queue.
- New arrivals and demoted processes join the tail of their destination queue.
- A process in Q0, Q1, or Q2 that uses its entire quantum and remains unfinished moves down one level.
- A process that finishes exits immediately, including when it finishes exactly as its quantum expires.
- If arrival times are equal, use the order listed in the table: P1 before P2.
- Assume no I/O, no aging or priority boosts, and zero context-switch overhead.
Process Arrival time CPU burst
P1 0 ms 8 ms
P2 0 ms 3 ms
P3 1 ms 5 ms
- Tasks
- Draw the Gantt chart.
- Show each process's queue movements and completion time.
- Identify which process finishes in the highest-priority queue and explain why it finishes first in this example. **Speaker Notes (Under Slide):**
(No additional speaker notes)

Slide 140

Slide Content:
(Visual Slide)

Answer: Multilevel Feedback Queue Scheduling

Step 1: Schedule processes in Q0 — quantum = 1 ms
At time 0, P1 and P2 arrive. The tie-breaking rule places P1 first. P3 arrives at time 1 and joins Q0 behind P2.

Time	Process	Remaining CPU time	Result
0–1	P1	7 ms	Moves to Q1
1–2	P2	2 ms	Moves to Q1
2–3	P3	4 ms	Moves to Q1

Step 2: Q0 is empty; schedule Q1 — quantum = 2 ms
The queue order is P1, P2, P3.

Time	Process	Remaining CPU time	Result
3–5	P1	5 ms	Moves to Q2
5–7	P2	0 ms	Finishes
7–9	P3	2 ms	Moves to Q2

Step 3: Q0 and Q1 are empty; schedule Q2 — quantum = 4 ms
The queue order is P1, P3.

Time	Process	Remaining CPU time	Result
9–13	P1	1 ms	Moves to Q3
13–15	P3	0 ms	Finishes

Speaker Notes (Under Slide):
(No additional speaker notes)

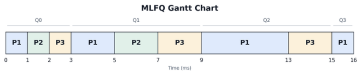
Slide 141

Slide Content:
(Visual Slide)

Step 4: Only Q3 contains a ready process

P1 runs from **15–16 ms**, completing its final 1 ms.

Gantt chart :



Queue movements and completion times

Process	Queue movement	Completion time
P1	Q0 → Q1 → Q2 → Q3 → Finished	**16 ms**
P2	Q0 → Q1 → Finished	**7 ms**
P3	Q0 → Q1 → Q2 → Finished	**15 ms**

Which process finishes in the highest-priority queue?
P2 finishes in Q1. Its 3 ms burst requires only **1 ms in Q0 + 2 ms in Q1**.
P2 finishes first in this example because it completes using the CPU time allocated in the upper queues. P1 and P3 still need CPU time after Q1 and must continue in lower-priority queues.

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 142

Slide Content:
2.11 ADVANCED
A feedback scheduler adapts priority from observed CPU use
79
New job
enter high queue
Short burst
finish quickly
Uses quantum
move downward
Waits too long
age upward → Dispatch: highest nonempty queue
Queue count, quantum sizes, promotion, and demotion rules define the policy.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide shows how a feedback scheduler changes priorities: new jobs start high, jobs that use their full CPU allocation

move down, and jobs that wait too long can move up.

Slide 143

Slide Content:
2.11 ADVANCED
Multiprocessor scheduling must place work as well as order it
80
GLOBAL QUEUE
shared ready work
PER-CPU QUEUE
local scheduling
LOAD BALANCE
move work among CPUs
AFFINITY
keep cache locality
NUMA AWARENESS
keep work near memory
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide says that with multiple CPUs or cores, the OS must decide both which process or thread runs next and where it runs.

Slide 144

Slide Content:
TABLE

Concept — Meaning — Why it matters
Global queue — CPUs select tasks from one shared ready queue. — Available CPUs can take available work, but access to the shared queue requires coordination.
Per-CPU queue — Each CPU has its own ready queue. — Scheduling can happen locally, but some CPUs may have more work than others.
Load balancing — Redistribute tasks among CPUs. — Avoid having one CPU overloaded while another is idle.
Processor affinity — Prefer or require a task to run on a particular CPU or set of CPUs. — Returning to the same CPU can reuse instructions and data already in its cache.
NUMA awareness — Schedule tasks near the main memory containing their data. — On NUMA systems, accessing local memory is generally faster than accessing another node's memory. **Speaker Notes (Under Slide):**

With one CPU, scheduling mainly answers "Who runs next?" With multiple CPUs, it also answers "Which CPU should run this task?"
Example: CPU 0 has four ready tasks, while CPU 1 is idle. Moving a task to CPU 1 can reduce waiting and let both CPUs work.
However, moving that task may have a cost:
Its useful data may already be in CPU 0's cache.
On a NUMA system, its data may be in memory local to CPU 0's node.
The scheduler therefore balances spreading the work against preserving efficient access to data.
NUMA stands for Non-Uniform Memory Access: memory-access speed depends on which processor accesses which memory location.

Slide 145

Slide Content:
2.11 ADVANCED
NUMA makes CPU placement a memory-latency decision
81
CPU PACKAGE 0
cores 0-3 • shared cache

LOCAL MEMORY 0
lower latency for package 0
INTERCONNECT
remote access costs more
CPU PACKAGE 1
cores 4-7 • shared cache
LOCAL MEMORY 1
lower latency for package 1
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Affinity helps most when thread placement and memory placement stay aligned. **Speaker Notes (Under Slide):**

Ask: If a thread moves from package 0 to package 1 but its pages remain on node 0, what changes? Expected: load may improve, but remote-memory latency and cache misses may increase.
In NUMA, a node is usually a group of CPU cores together with the RAM that is local to those cores.

Slide 146

Slide Content:
2.11 ADVANCED
Load balancing can conflict with processor affinity
82
LOAD BALANCING
Avoid idle processors
Push or pull runnable work
Improves aggregate utilization
PROCESSOR AFFINITY
Prefer the same processor
Preserves warm caches and locality
Can be soft or hard
Moving a task may balance load but destroy useful locality.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains a trade-off: moving tasks can keep more CPUs busy, while keeping tasks on the same CPU can preserve faster access to cached data.
Load balancing distributes runnable tasks across CPUs so that one CPU is not overloaded while another is idle.
Push: Work is moved from a busy CPU's queue to a less busy CPU.
Pull: An idle or less busy CPU takes work from another CPU's queue.
Aggregate utilization: How effectively the CPUs are being used overall.
Processor affinity keeps a task on the same CPU, or within a selected group of CPUs.
The benefit is a warm cache: the CPU's cache already contains useful instructions and data from the task's previous execution. "Warm" refers to useful cached content, not temperature.
Soft affinityThe scheduler prefers the same CPU but may move the task when beneficial.
Hard affinityThe task is restricted to a specified CPU or set of CPUs until that restriction changes.

Slide 147

Slide Content:
2.11 ADVANCED
Real-time scheduling focuses on meeting timing constraints
83
SOFT REAL TIME
Occasional misses degrade quality
Examples: media and interactive control
Best effort with priority
HARD REAL TIME
A missed deadline is system failure
Requires bounded execution and blocking
Needs analyzable admission

Real-time correctness includes both the result and when it arrives.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains that real-time scheduling aims to finish tasks before their deadlines, and that missing a deadline has different consequences in soft and hard real-time systems.
A deadline is the latest time by which a task must finish. Real-time does not simply mean "very fast"–it means finishing within the required time.
Soft real-time :
Meet deadlines, but some misses are acceptable.
If a deadline is missedQuality or responsiveness decreases.
ExampleA video call: a late frame may cause a brief freeze.
Hard real-time:
Every required deadline must be met under the system's specified operating conditions.
If a deadline is missed The system has violated a correctness requirement.
Example: A machine's safety controller: a stop command must arrive before a specified deadline.

Slide 148

Slide Content:
2.11 ADVANCED
RMS uses fixed priorities; EDF uses nearest deadline
84
RATE-MONOTONIC
Fixed priority
Shorter period → higher priority
Analyzable periodic tasks
EARLIEST-DEADLINE-FIRST
Dynamic priority
Nearest deadline runs first
Higher possible utilization (up to 100%)
Assumptions about periods, deadlines, and execution time determine schedulability.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares two real-time scheduling algorithms: RMS gives higher priority to tasks that repeat more frequently, while EDF runs the ready job with the earliest deadline.

Slide 149

Slide Content:
2.11 LINUX CONTEXT
Linux fair scheduling is moving from CFS toward EEVDF
85
CFS FOUNDATION
• Models an ideal fair CPU
• Tracks normalized virtual runtime
• Historically selects the task with smallest vruntime
EEVDF DIRECTION
• Linux began transitioning in kernel 6.6
• Eligible tasks have nonnegative lag
• Earliest virtual deadline runs next
Both pursue fairness; EEVDF adds deadline-based responsiveness for eligible tasks.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide explains how Linux shares CPU time fairly and how EEVDF improves responsiveness while maintaining fairness.
CFS means Completely Fair Scheduler. Its basic idea is to give each runnable task its fair share of CPU time.
EEVDF means Earliest Eligible Virtual Deadline First. Linux began its transition toward EEVDF in kernel 6.6. It makes the scheduling decision in two steps:
Check eligibility. Is the task owed CPU time, or exactly caught up with its fair share?

Compare virtual deadlines. Among eligible tasks, select the earliest virtual deadline.

Slide 150

Slide Content:
SYNTHESIS
Chapter 2 connects execution, coordination, and scheduling
86
Process
owns resources
Thread
executes within process
IPC
coordinates processes
Scheduler
selects the next runnable entity
Next: synchronization, race conditions, and safe coordination among concurrent activities.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Process – owns resources: A process provides the resources needed by a running program, such as its address space and open files.
Thread – executes within a process : A thread executes instructions. Threads in the same process share many resources, but each has its own execution state and stack.
IPC – coordinates processes : Interprocess Communication lets processes exchange information using mechanisms such as pipes, message passing, and shared memory.
Scheduler – selects the next runnable entity : The scheduler chooses which ready task gets CPU time next–typically a thread in modern operating systems.

Slide 151

Slide Content:
End of Chapter 2 **Speaker Notes (Under Slide):**

(No additional speaker notes)
